KSS Pre-Test — 오답노트 + Prep

KSS Pre-Test 오답노트

모의고사에서 틀리거나 놓친 내용만 모아 둔 노트다.
각 항목은 시험장에서 말로 설명할 수 있을 정도로, 짧은 교과서처럼 풀어 썼다.
맨 아래 **직전 30초**만 압축 복습용이다. 3회 오답은 풀고 나서 이어서 추가한다.

1회 모의

1. 이름 있는 rvalue 참조 변수는 lvalue다

함수 매개변수가 `int&&`나 `std::string&&`처럼 rvalue 참조로 선언되어 있어도, 그 매개변수에 **이름이 붙어 있으면** 함수 안에서 그 이름을 쓴 식은 **lvalue**로 취급된다.
값 범주는 "선언에 `&&`가 붙어 있나"가 아니라, **그 순간 식이 이름 있는 객체를 가리키는지**로 결정되기 때문이다.

그래서 `void f(T&& x)` 안에서 `x`를 그대로 다른 함수에 넘기면 이동이 나가지 않는 경우가 많다.
다시 rvalue로 취급하려면 `std::move(x)` 또는 `std::forward<T>(x)`가 필요하다.

시험에서는 예를 들어 `f(3)`을 호출했을 때, 추론된 `T`는 `int`, 매개변수 `x`의 타입은 `int&&`이지만, 함수 본문의 이름 `x`의 값 범주는 **lvalue**라고 써야 한다.

---

2. const 객체에 move를 해도 대개 복사다

`std::move`는 실제로 데이터를 옮기지 않는다. 인자를 **rvalue 참조로 캐스팅**할 뿐이다.
`const std::string cs`에 `std::move(cs)`를 하면 결과는 대략 `const std::string&&`가 된다. 캐스트 자체는 된다.

다만 일반적인 이동 생성자는 `string(string&&)`처럼 **const가 아닌** rvalue만 받는다.
`const string&&`는 그 생성자에 맞지 않으므로, 대신 `string(const string&)` **복사 생성자**가 선택된다.

정리하면, "const라서 rvalue로 변환이 안 된다"가 아니라, **변환은 되는데 이동 생성자가 const rvalue를 거절해서 복사로 떨어진다**가 정확한 설명이다.

---

3. 로컬 변수를 return할 때 move를 붙이지 않는다

함수 안에서 만든 로컬 `std::string local`을 반환할 때는 `return local;`이 정석이다.
컴파일러는 반환값 최적화(NRVO)로, 호출부가 받을 자리에 로컬을 처음부터 만들어 **복사·이동을 생략**할 수 있다.

여기에 `return std::move(local);`을 쓰면 그 최적화를 포기하고 이동 생성을 강제하는 경우가 많아, 성능이 오히려 나빠질 수 있다.

반대로, 값으로 받은 **매개변수**나 **멤버**를 반환할 때는 NRVO 대상이 아닌 경우가 많아서, 소유권을 넘기려면 `return std::move(other);`가 이득인 경우가 많다.

---

4. 소멸자만 정의한 raw 버퍼는 복사 시 double free가 난다

클래스가 `new char[]`로 버퍼를 소유하고 소멸자만 직접 정의한 채, 복사 생성·대입은 손대지 않으면, 컴파일러가 만드는 복사는 포인터 주소만 복사하는 **얕은 복사**다.
복사본과 원본이 같은 메모리를 가리키다가 각각 소멸하면서 **같은 주소를 두 번 해제**하게 된다. 이것이 double free이며 미정의 동작이다.

처방은 `std::vector`나 `std::unique_ptr` 같은 표준 타입에 소유를 맡겨, 특수 멤버를 직접 쓰지 않는 것(Rule of 0)이 가장 흔하다.
직접 소유해야 한다면 깊은 복사와 이동을 세트로 맞추거나, 복사를 삭제하고 이동만 허용한다.

---

5. malloc/free와 new/delete의 역할이 다르다

`malloc`과 `free`는 **메모리 블록의 할당과 해제만** 관여한다. 객체 생성자나 소멸자는 호출하지 않는다.

`new`와 `delete`는 메모리뿐만 아니라 **객체의 수명**까지 관여한다. `new`는 할당 뒤 생성자를 호출하고, `delete`는 소멸자 호출 뒤 메모리를 돌려준다.

짝을 맞춰야 한다. `new`로 만든 것은 `delete`, `new[]`로 만든 것은 `delete[]`, `malloc`으로 받은 것은 `free`다.
짝이 틀리면 미정의 동작이다.

6–7. x64 Windows 호출 규약

`x86에서는 cdecl은 스택을 caller가 정리하며 주로 C 라이브러리 계열이 사용하고, stdcall은 callee가 스택을 정리하며 WinAPI에서 사용됨. x64는 파라미터를 레지스터(rcx, rdx, r8, r9)를 통해 전달하며 스택은 caller가 정리한다.`

일반적인 C/C++ 함수에서 정수나 포인터 인자 앞 네 개는 대략 **RCX, RDX, R8, R9** 레지스터 순서로 전달되고, 그다음 인자는 스택을 쓴다.
추가로, 호출 전에 스택에 약 32바이트 shadow space를 남겨 두는 관례가 있다는 정도만 알면 가점이다.

함수 포인터로 등록한 콜백의 호출 규약이 호출 측과 맞지 않으면, 인자가 잘못된 자리에 들어가거나 스택 정리가 어긋나 **크래시나 이상한 동작**으로 이어질 수 있다.

---

8. new char[len] 뒤에 strcpy하면 버퍼를 넘친다

`strlen(s)`가 `len`일 때 `new char[len]`만 하면, 문자열을 넣는 데 필요한 **널 문자(\0) 한 자리가 없다**.
`strcpy`는 널까지 쓰므로 할당 범위를 한 바이트 넘어가고, 힙 버퍼 오버플로(미정의 동작)가 된다.

할당할 때는 `new char[len + 1]`처럼 널 자리를 포함해야 한다.

---

9. 얕은 복사와 스레드 값 전달은 double free로 이어진다

복사 생성자가 포인터만 복사하면, `std::thread(worker, a)`처럼 객체를 값으로 넘길 때 **같은 버퍼를 가리키는 복사본**이 또 생긴다.
스레드 쪽 객체와 원본이 각각 소멸자에서 해제하면 동일 주소를 두 번 지우게 된다.

시험 한 줄: 얕은 복사라면 **깊게 복사하거나, 복사를 막거나, 표준/스마트 포인터에 소유를 맡긴다.**

---

10. append에서 흔한 두 실수

새 버퍼를 할당한 뒤 `data = p`만 하고 이전 `data`를 해제하지 않으면 **메모리 누수**다.
또한 `len`을 새 길이에 맞게 갱신하지 않으면, 이후 크기 계산이 틀어져 또 다른 오버플로로 이어질 수 있다.

```
// 잘못된 예
void append(const char* s) {
  char* p = new char[len + strlen(s) + 1];
  strcpy(p, data);
  strcat(p, s);
  data = p;   // 옛 data 미해제 → 누수
  // len 미갱신 → 다음 append 크기 틀림
}

// 올바른 감
void append(const char* s) {
  int add = (int)strlen(s);
  char* p = new char[len + add + 1];
  strcpy(p, data);
  strcat(p, s);
  delete[] data;   // 옛 버퍼 해제
  data = p;
  len += add;     // 길이 갱신
}
```

---

11. new[]에는 delete[]를 써야 한다

배열 형태로 `new[]`한 메모리는 **`delete[]`**로 해제해야 한다.
`delete`만 쓰면 미정의 동작이다.

12–14. g_last, g_count, data race를 구분한다

`join` **이전**에 worker가 `g_last`(원본 객체)에 `append`하고, main이 같은 객체의 문자열을 읽으면, 동기화 없는 충돌 접근이므로 **data race**다.

`g_last` 같은 raw 정적 포인터는 객체가 사라진 뒤에도 주소를 들고 있을 수 있어 **dangling** 위험이 있다.

반면 `g_count`처럼 **한 스레드만 쓰고**, main은 **`join` 이후에만** 읽는다면, 그 코드만 놓고 보면 data race가 아니다.
공유 변수라고 해서 무조건 race라고 단정하면 안 된다. worker가 여러 개면 이야기가 달라진다.

15–22. CreateFile / ReadFile 리뷰 포인트

`CreateFile`이 실패하면 반환값은 **`INVALID_HANDLE_VALUE`**이다. NULL이 아니므로 `if (!h)` 같은 검사는 실패를 놓친다.

`outBuf[read - 1] = '\0'`은 위험하다. `read`가 0이면 음수 인덱스에 가깝게 접근해 미정의 동작이고, 0이 아니어도 **마지막에 읽은 데이터 한 바이트를 널로 덮어써서** 내용을 망가뜨린다.
널 종료가 필요하면 보통 버퍼 크기보다 하나 적게 읽고(`outSize - 1`), `outBuf[read] = '\0'`처럼 **읽은 길이 다음 칸**에 널을 넣는다.

`ReadFile` 한 번이 파일 전체를 읽었다고 보장하지 않는다. **부분 읽기**를 가정하고 루프나 길이 검사가 필요하다.

공유 모드를 0으로 두면 다른 열기와의 공유가 막혀 호환성 문제가 날 수 있다.
경로가 사용자 입력이면 상대경로나 `..`을 이용한 **path traversal**을 의심하고 검증해야 한다.

`GetLastError`는 API가 **실패한 직후**에 호출해야 의미가 있다.
한글 경로 등을 고려하면 `CreateFileA`보다 `CreateFileW`가 더 안전한 경우가 많다.

23–24. 후킹·보안 서술의 뼈대

보안 서술 문제는 기법 나열보다 다음 순서가 점수를 준다.
원리 → 탐지 또는 완화 방법 → 우회 가능성 → 오탐·성능 → 서버나 다계층.

클라이언트 후킹 탐지는 “완벽하게 막는 장치”가 아니라 **서버로 올리는 신호**에 가깝다.
서버는 여러 신호를 모아 판단하고, 중요한 게임 결과는 서버가 권위를 갖는다.

클라이언트는 공격자의 손에 있으므로, 클라만으로 완벽한 차단을 약속하는 답은 약하다.

코드리뷰를 쓸 때의 습관

완벽한 재작성보다, 이슈를 번호로 나열하는 편이 점수에 유리하다.
각 이슈마다 **무엇이 잘못됐는지, 영향(미정의 동작·크래시·레이스·누수), 고치는 방향**을 한두 줄로 적는다.

항상 네 가지 렌즈로 훑는다. **정확성, 수명·소유권, 보안, 동시성·성능.**

특히 자주 나오는 네 덩어리는 다음과 같다.
버퍼 크기와 널 자리, 얕은 복사와 `delete[]`, CreateFile의 `INVALID_HANDLE_VALUE`, `join` 이전의 공유 접근.

2회 모의

25. auto는 top-level const를 벗기는 경우가 많다

`const int cx = 1;`일 때 `auto a = cx;`의 `a` 타입은 **`int`**다.
값으로 복사하면서 맨 바깥의 const가 추론에서 빠지기 때문이다.

`auto& b = cx;`처럼 참조로 받으면 `b`의 타입은 **`const int&`**가 되어, 원본의 const가 유지된다.

---

26. 원하지 않는 암시 변환은 explicit으로 막는다

`Path(const char*)` 생성자만 있으면, 의도와 다르게 다른 타입이 `Path`로 암시 변환되어 함수에 들어갈 수 있다.
생성자를 `explicit`으로 표시하면, 그런 암시 변환을 컴파일 단계에서 막을 수 있다.

---

27. 기반 포인터로 지울 때는 가상 소멸자가 필요하다

`Base* p = new Derived;` 뒤에 `delete p;`를 할 때, `Base`의 소멸자가 가상이 아니면 **파생 클래스 소멸자가 호출되지 않을 수 있다**.
이는 미정의 동작이고, 파생 쪽 자원이 안 풀리는 누수로 이어질 수 있다.

기반 클래스에 `virtual ~Base();`를 두는 것이 일반적인 수정이다.

---

28. const 멤버 함수 안에서 this는 const 객체 포인터다

`int get() const`처럼 멤버 함수에 const가 붙어 있으면, 그 안에서의 `this` 타입은 **`const S*`**다.
그래서 일반 멤버를 수정할 수 없다. `mutable`로 표시된 멤버만 예외적으로 수정할 수 있다.

---

29. malloc으로 받은 주소는 free로만 해제한다

`malloc`이 돌려준 포인터에 `delete`나 `delete[]`를 쓰면 미정의 동작이다.
반드시 **`free`**와 짝을 맞춰야 한다. new/delete 짝과 섞지 않는다.

---

30. rvalue를 forward로 두 번 넘기면 두 번째는 moved-from이다

`std::forward`로 같은 rvalue를 두 번 넘기면, 첫 번째에서 이미 이동이 일어날 수 있다.
두 번째 호출에는 **옮겨진 뒤의 객체(moved-from)** 가 전달되어, 결과가 위험하거나 의미가 없다.
컴파일 에러가 난다고 단정할 수는 없다.
완벽 전달은 보통 **한 번**만 한다.

---

31. 스레드와 vector를 같이 쓰면 join 전에 읽지 않는다

한 스레드가 `vector`에 `push_back`하는 동안, 다른 쪽에서 `join` 전에 `size`나 인덱스로 읽으면 **data race**이고 미정의 동작이다.
재할당 중에 읽으면 더 위험하다.

고치는 방법은 단순하다. **`join`이 끝난 뒤에만** 읽거나, 공유 접근을 **mutex**로 보호한다.
`vector` 자체에 `atomic`을 붙이는 식으로는 해결되지 않는다.

이 문제 코드처럼 출력 루프만 `join` 뒤로 옮기면, 그 패턴의 race는 대부분 사라진다.

---

32. 실행과 쓰기를 한 페이지에 동시에 주지 않는다

`PAGE_EXECUTE_READWRITE`는 한 메모리 구역에 **쓰기와 실행**을 동시에 허용한다.
코드 주입이나 런타임 패치에 유리해져 보안상 매우 위험하다.

가능하면 쓰기 권한으로 내용을 넣은 뒤, `VirtualProtect` 등으로 **실행 권한만** 남기는 식으로 권한을 분리한다.

---

33. VirtualAlloc 리뷰에서 무엇을 말할지

코드에 바로 보이는 문제는 W+X, 크기 미검증, 실패 시 `out` 처리 부족이다.
`size_t` 크기는 부호가 없으므로, “음수”가 들어오면 거대한 양수로 래핑될 수 있어 검증이 필요하다.

할당에 성공한 뒤의 해제는 보통 **호출자의 책임**이다. 할당 함수 안에서 바로 `VirtualFree`를 하라는 뜻이 아니다.
문제에 해제 코드가 아예 없다면, 해제 미언급만으로 크게 감점할 이슈는 아니다.
다만 “이 버퍼의 소유·해제 책임이 호출 측에 있다”는 한 줄은 가점이 될 수 있다.

2회에서 이미 잘 맞춘 것

이름 있는 `string&&`를 `sink`에 그냥 넘기면 lvalue라는 점,
`move`를 붙여야 rvalue가 된다는 점,
`forward`를 두 번 쓰는 위험,
cdecl은 호출자가·stdcall은 피호출자가 스택을 정리한다는 점,
코드 체크섬과 서버 검증·유저모드 한계를 말한 점은 유지하면 된다.

3회 모의

size_t 덧셈 오버플로 (Q2 공백)

`size_t need = user_a + user_b;` 후 `if (need < cap)`는 **항상 안전하지 않다**.
`size_t` 덧셈이 넘치면(wrap) `need`가 작은 값이 되어 검사를 통과한 뒤, 실제로는 거대한 할당·복사를 하려다 사고 난다.

더 안전한 패턴 예:

```
if (user_a < cap && user_b <= cap - user_a) { ... }
```

(덧셈을 하기 전에 남는 용량과 비교한다.)

---

unique_ptr는 복사되지 않는다 (Q5)

`Holder`가 `unique_ptr`를 멤버로 두면, `Holder b = a;`처럼 **복사**하려 하면 컴파일이 되지 않는다.
`unique_ptr`는 복사가 삭제되어 있고, 그에 따라 `Holder`의 복사도 삭제되기 때문이다.

`Holder c = std::move(a);`는 **이동**이라 컴파일된다.
이때 소유권이 `c`로 넘어가고 `a.p`는 비는 것이지, 깊은 복사가 아니다.

시험 한 줄: unique_ptr → **복사 불가, 이동만**. (얕은/깊은 복사 프레임으로 설명하면 감점.)

널 종료 위치 (Q1 보완)

`char buf[8]; copy_n(buf, user, user_len);`에서 `dst[n] = '\0'`이면, `n == 8`일 때 `dst[8]`은 **버퍼 밖**이다.
고칠 때는 `dst[n-1]`로 바꾸는 것이 답이 아니다. 그건 마지막 데이터를 널로 덮거나, `n==0`에서 또 위험하다.

올바른 감: **`n < 버퍼크기`(보통 `n <= size-1`)인지 먼저 검사**한 뒤, 허용된 `n`에 대해 `dst[n] = '\0'`을 쓴다.
또는 최대 `size-1`바이트만 복사하고 그다음 칸에 널을 둔다.

Packet 리뷰에서 더 말할 것 (Q8)

이미 말한 것(대입 시 옛 buf 미해제·누수, shallow로 double free, put 길이 검사)은 맞다.

추가로 찍으면 좋은 것:

- `send(Packet p)` 값 전달 → **또 shallow 복사** → double free 악화
- `cap == 4`인데 `put("ping")` → `"ping"`+널 = 5바이트 → **이미 OOB**
- `b.put("toolong!!")` → 용량 초과 OOB
- 자기대입 `operator=` 미처리
- Rule of 3/5 미비 (복사는 얕은데 소멸은 `delete[]`)

DllMain 금지·위험 (Q9 핵심)

문제의 본진은 "핸들을 안 들고 있다"보다 **DllMain 안에서 하면 안 되는 일**이다.

1. **`LoadLibrary`를 DllMain에서 호출** → loader lock 데드락·로더 재진입 위험.
2. **`CreateThread`도 DllMain에서 비권장** — 로더·동기화와 충돌할 수 있다.
3. 캡처 있는/없는 **람다를 스레드 프로시저로 넘기는 것**은 컴파일·호출 규약상 위험하거나 불가인 경우가 많다.
4. 만든 스레드 **핸들을 CloseHandle하지 않으면** 누수.
5. 초기화는 DllMain이 아니라 **별도 init API**에서 하는 편이 안전하다.
6. (가점) `LoadLibrary` 성공 여부 미검사 — 이건 보조 지적.

3회에서 잘한 것

RVO/NRVO와 `return move(local)` 방해,
`f(x)`/`f(move(x))`의 T·타입과 이름 있는 변수=lvalue,
const vs constexpr + rand,
C++ 예외 vs SEH,
안티디버그 단서(IsDebuggerPresent·타이밍·0xCC·DR)와 우회,
즉시 종료만으로는 부족하고 서버·다신호로 간다는 태도.

직전 30초 압축

이름 있는 `&&` 변수는 lvalue다.
const에 move하면 대개 복사다.
로컬은 `return local`이고, forward는 한 번이다.
얕은 복사는 double free로 가고, vector나 unique_ptr에 맡긴다.
unique_ptr는 복사 불가·이동만 (얕은/깊은 복사가 아님).
auto 값 복사는 top-level const를 벗긴다. explicit, 가상 소멸자를 기억한다.

size_t a+b는 wrap → 덧셈 전 비교.
x64 인자는 RCX, RDX, R8, R9다. cdecl은 caller, stdcall은 callee가 스택을 정리한다.
`new[len]`+strcpy는 널 자리 때문에 오버플로이고, new[]는 delete[], malloc은 free다.
널은 `dst[n-1]`이 아니라 **공간 확인 후 `dst[n]`**.
join 전 공유는 race다. vector는 atomic으로 안 고치고, mutex나 join 후 읽기다.
CreateFile 실패는 INVALID_HANDLE_VALUE다.
DllMain에서 LoadLibrary/CreateThread 금지(loader lock).
W+X는 피하고 크기를 검증한다.
후킹·안티디버그는 우회를 가정하고 서버·다계층과 연계한다.

KSS Pre-Test Prep

KSS Anti-Cheat 직무테스트(Pre-Test) 준비 노트

대상: KRAFTON KSS — Anti-Cheat / Client Security
기준: 경력 · **C++11 Modern 최소** (rvalue/`move`/`forward`) + Windows/보안
시험: **목요일** · 현직자: TMP↓ · **C++ 메인** · 리버싱/드라이버=우대

⏱ 시험 직전 1시간 — 여기만 보세요

규칙: 아래 표 순서대로만. PART B 전체·PART E·부록·모의고사·새 개념 **금지**.
본문에 ⏱ 직전1h 표시가 있는 절 = 이 루트에 포함.
시간이 20분뿐이면: **치트시트 → `1.5`–`1.6b` → PART D** 만.

분	무엇을	어디로 · 읽는 법 (`Ctrl+F`로 ⏱ 또는 절 번호)
0–5	치트시트	`##` 치트시트 — 소리내어
5–25	move·forward·참조	`###` `1.5` → `1.5b` → `1.6` → `1.6b` — **표·한 줄·암기 치트만** (M1~M3 재풀이 X)
25–33	const	`###` `1.4` 표 + `1.4b`의 **우측값·move / 면접 한 줄**
33–40	new·volatile	`###` `1.2` · `1.3` — 한 줄·표만
40–48	리뷰 렌즈	`###` `1.9` · `1.10` · `##` `PART D`
48–54	보강·IPC	`###` `1.11` 한 줄 모음 · `###` `3.2b` 표
54–58	보안·호출	`###` `1.7` 표 · `1.8` 또는 `1.1`(우회·오탐) · `1.12` **§0만**
58–60	복창	정의→차이→함정 / 렌즈: 보안·정확성·성능·Windows

직전 1시간에 열지 말 것: PART B(B1~B4 전체) · PART E · 부록 Sir.LOIN · Mock 2·3회 · Windows Internals · SFINAE.

바로가기 (상황별)

상황	이동
시험 직전 1시간	위 표 **처음부터**
오늘 공부(수 D-1 등)	3일 스케줄
모의고사	1회 · 2회 · 3회
기출·개념 본진	PART A (⏱ = 직전1h 포함)
체크리스트·구술 연습	PART B ← 직전1h **제외**
Win/IPC/공고	PART C · IPC만 직전1h
리뷰 렌즈만	PART D
서류·면접	PART E ← 직전1h **제외**
현직자 우선순위	시험이 뭘 보나

문서 지도 (읽기 순서 · 평소)

```
[직전 1h 루트] ── 시험 당일만
      │
PART A 기출·특집 (1.1~1.12)  ← 평소·수요일 본진
      │
PART B 체크리스트            ← 구술 연습 (직전 X)
PART C Win/OS/IPC            ← 평소; 직전은 3.2b만
PART D 리뷰 렌즈             ← 직전 포함
PART E · 부록                ← 면접용 (직전 X)
```

주제 → 위치

주제	절	직전1h
move / forward / 참조 / RVO	A 1.5–1.6b	✅
const	A 1.4, 1.4b	✅
malloc/new, volatile	A 1.2–1.3	✅
CreateFile · 클래스 리뷰	A 1.9–1.10, D	✅
보강(정수·문자열·…)	A 1.11	✅ 한줄
IPC 파이프/공유메모리	C 3.2b	✅ 표
호출규약 · 핵/안티디버그	A 1.7–1.8, 1.1	✅ 짧게

개편 뼈대	A 1.12 §0	✅ §0만
Q&A 전체 · C++03 · 서류	B, E, 부록	❌

이 문서 읽는 법

1. **시험 직전:** 맨 위 ⏱ **1시간 루트만**
2. **수 D-1까지:** 스케줄 → PART A (⏱ 절 우선) → PART D
3. **막힐 때:** PART A 해당 절 + PART B 같은 주제
4. **면접 전:** PART E · 부록 · C (드라이버는 우대만)
5. A와 B가 겹치면 → **답변은 A 우선**

3일 스케줄 (목요일 시험)

> 기준: **월(D-3) · 화(D-2) · 수(D-1) → 목 시험**
> 원칙: 새 범위 욕심 금지. **PART A 기출 C++ + 코드리뷰**가 합격 축.

우선순위 (시간에 쫓길 때) — 현직자 반영

1. **C++11 Modern** — `1.5–1.6b` (rvalue / `move` / `forward` / 참조축소) + `1.4b` const + `1.5b` RVO ← **최우선**
2. **코드리뷰 C++** — `1.10` 클래스(소유권·Rule5) + `1.11` 중 정수/C문자열/가상소멸 ← 필수
3. **Win32 리뷰** — `1.9` CreateFile (반환값·크기·널·HANDLE) ← 필수
4. `1.2–1.4`, `1.7–1.8`, `1.1` — 정의·차이·함정만 짧게
5. 드라이버·리버싱·TMP·PART E 심화 ← **수요일 이후 금지** (우대는 면접용)

Day 1 — 월요일 (D-3) · C++ 몰입

블록	시간감	할 일
오전 1	90분	PART A `1.2–1.4` (malloc/new, volatile, const/constexpr) — 정의+차이+함정
오전 2	90분	PART A `1.5–1.6b` (move/forward/참조축소/함정) — 표 암기 후 소리 내어 설명
오후 1	90분	PART B `B1` A급 구술 + `B0` Rule of 5 / smart pointer
오후 2	90분	클래스 리뷰 2문제 + PART B `B1` I05 손코딩 (답 보기 전 15분)
밤	30분	치트시트 C++ 줄만 · A급 ×만 재확인

월 종료 기준: B1 A급 막힘 없이 + move/forward/참조축소/volatile 설명 가능.

Day 2 — 화요일 (D-2) · 코드리뷰 + 윈도우/보안

블록	시간감	할 일
오전 1	90분	PART A 1.9 CreateFile 체크리스트 → 유사 Win32 코드 1개 리뷰
오전 2	60분	PART A 1.7 호출규약 + PART B B3 빌드/링크 + 부록 OS★
오후 1	90분	PART A 1.1 안티디버깅, 1.8 핵 방어 + 부록 암호/캐시 — 우회·오탐 포함 모의답
오후 2	90분	클래스 리뷰 1 + CreateFile형 1. PART B B·C·D·E 중 ×만 구술
밤	45분	PART B I01 RAII 핸들 + C2 용어 · PPT 05 3분 구술

화 종료 기준: 코드리뷰 이슈 5개+ · A급 유지.

Day 3 — 수요일 (D-1) · C++ 재고정 + 모의 + 개편 훑기

목표: **기출·C++11 본진** + **1.12는 한 번 훑기(암기 X)**. 심화 새 판 금지.

블록	시간감	할 일	보지 말 것
오전 1	60–75분	**1.5–1.6b만** 다시: 표 보고 → 가리고 구술. M1~M3·const+move·이름 있는 &&·double-forward	SFINAE, 커널 구현
오전 2	45분	**1.4b const** + **1.5b RVO** 한 바퀴	—
오전 3	**60분**	모의 **1회** (노트 금지) → 채점. 여유·취약하면 2·3회는 저녁/목 전	풀이 중 답지 금지
점심 후	60분	틀린 C++ 재구술 + **클래스 리뷰 1** + CreateFile 체크리스트 구술	PART E, PPT
오후 1	30–40분	**1.11 빠른 훑기** (정수·문자열·가상소멸·INVALID·atomic)	—
오후 2	**30–40분**	### **1.12 개편 대비 소재 풀 통독** + 보안 뼈대로 **후킹 방지** 가정 답 1회 구술	각 항목 딥다이브·새 자료
저녁	25분	치트시트 + B1 A급 □	새 노트 추가 금지

밤	—	수면	—

수 종료 기준 (합격축)

- [ ] `move` / `forward` / 참조축소 / 이름 있는 `T&&`=lvalue / const+move≈copy
- [ ] 클래스·CreateFile 리뷰 렌즈
- [ ] volatile≠atomic, malloc vs new
- [ ] `1.12` **한 번 읽음** + 보안 답 뼈대(원리→탐지/방어→우회→오탐·성능) 말로 1회

Day 4 — 목요일 (시험일)

시점	할 일
시험 전 1시간	문서 맨 위 **「⏱ 시험 직전 1시간」** 표만 따라가기
시간이 20분뿐	치트시트 + `1.5–1.6b` 표 + PART D
시험 중	**정의 → 차이 → 함정 → (가능하면 AC 한 줄)**. 막히면 리뷰 렌즈
쓰지 말 것	PART B/E/부록 새로 읽기, 드라이버·SFINAE

하지 말 것

- Internals 통독, 새 드라이버 실습, SOLID/Sir.LOIN 전체
- **SFINAE·템플릿 메타프로그래밍 심화** (현직자: 크게 중요 X)
- 리버싱/커널을 C++보다 위에 두기 (우대≠필수, **메인은 C++ 개발**)
- 수요일 밤 새 문제 대량 추가

일일 최소 통과선

요일	최소
월	B1 A급 + move/forward/참조축소 + 클래스리뷰 1회
화	CreateFile 체크리스트 + 호출규약 + 핵방어 모의답 1개
수	move/forward 재구술 + C++ 기출 모의 + 클래스/CreateFile 렌즈 (새 주제 0)

치트시트 (시험 직전)

> ⏱ **직전1h · 0–5분** · 전부 소리내어 읽고 다음 절로.

```
volatile ≠ atomic
move = cast to rvalue ref, 이동 자체 아님
forward = 값 범주 유지 전달
참조축소: & 하나라도 있으면 &
이름 있는 T&& 변수 = lvalue / const+move ≈ copy
로컬 return move(local) 금지 · 파라미터·멤버 move는 OK
```

```
const = Constant 자물쇠 / constexpr = Expression을 컴파일에 굳힘
malloc = 메모리 / new = 메모리+수명 · new[]↔delete[]
x64 Win: RCX RDX R8 R9 + 32B shadow
CreateFile 실패 = INVALID_HANDLE_VALUE (NULL 아님)
코드리뷰: 반환값·크기·수명·권한·부분I/O·복사·소유권
파이프=스트림·복사 / 공유메모리=매핑·별도동기화
훅·안티디버그: 우회 가정 + 오탐·성능 + 서버/다계층
```

시험이 뭘 보나

지난 Pre-Test는 대략 아래 3축이었습니다.

축	비중 추정	기출 예
Modern C++ / 언어 기초	높음	malloc/new, volatile, const/constexpr, move/forward, 참조 규칙, 클래스 코드 리뷰
Windows / 저수준 / 호출규약	중~높음	CreateFile/ReadFile 코드 리뷰, calling convention
보안·안티치트 사고력	중	Intel CPU 안티디버깅, 특정 함수 호출 핵 방어

떨어진 지점 가설: C++ 개념을 “써본 적 있음”이 아니라 **정의·차이·함정·성능·UB까지 말로 정확히** 설명해야 하는 시험.

준비 원칙

1. 각 개념을 **한 문장 정의 → 차이표 → 함정/미정의 동작(Undefined Behavior) → AC에서 왜 중요한지** 순으로 말할 수 있게 한다.
2. 코드 리뷰 문제는 **보안 / 정확성 / 성능 / 수명(RAII)** 네 렌즈로 훑는다.
3. 보안 문제는 **완벽한 방어**가 아니라 **탐지 지점·우회면·오탐·성능**을 같이 말하는 답이 가점.

★ 현직자 익명 답변으로 본 우선순위

> 출처: 현직자 익명 Q&A 요지 (예: 블라인드 172371 계열). 공고·우대와 맞춰 **공부 순서만** 고정할 때 사용.

구분	현직자 요지	이 노트에서
신입	**C++03 문법 정도** + 언어 개발 경험이 가장 중요. OS 등은 학부 개념이면 충분	B4 ★ + 기출 리뷰
경력 (본인)	**C++11 Modern** — 특히 **rvalue ref,** `std::move`**,** `std::forward`	PART A `1.5`–`1.6b` **최우선**

템플릿 메타	**SFINAE 같은 TMP는 크게 중요 X**	F05·심화 TMP = 스킵/맨 뒤
업무 메인	윈도우 **핵 탐지 솔루션** — 커널 드라이버 + **유저 라이브러리**, 메인은 **C++ 개발** (리버싱이 메인 아님)	Pre-Test도 C++/Win32 리뷰 중심
필수 감	C/C++, modern C++, OS 지식	PART A + C1 요약
우대	x86/64 asm, 커널 드라이버, x86 리버싱 — **우대일 뿐 필수 아님**	포트폴리오로 가점, 시험 전 심화 X
성향	로우레벨 흥미, C++ 선호, OS·핵 동작 파악하고 싶은 사람	답변에 "우회 가정·관측·OS 경계" 톤

경력 Pre-Test에 대한 해석

1. **합격 축 = C++11을 말로·코드리뷰로 정확히** (move/forward/참조/소유권/UB).
2. 윈도우 API·호출규약·간단한 안티디버그는 **기출에 나온 만큼만**.
3. 드라이버/리버싱/TMP로 점수 벌려 하지 말 것 — 면접·서류에서 우대 카드로 쓰면 됨.

PART A. 지난 기출 10문 (본진)

> 지난 Pre-Test와 가장 닮은 축. **공부 1순위.** 하위: `1.1`~`1.11 · **1.12` 개편 대비 소재 풀**(흩기)

1.1 Intel CPU에서 안티디버깅 방안

> ⏱ 직전1h · 54–58분(우회·오탐만, 기법 나열 X)

시험에 필요한 범위: 아래 **핵심 4(+2)** 만. 종류 나열 암기 X.
각 기법 = **원리 → 무엇을 탐지 → 우회(개념) → 오탐**.

전제: 디버거는 (1) attach (2) 정지/싱글스텝 (3) SW BP=`0xCC` 또는 HW BP=DR (4) 예외 가로채기. 안티디버깅은 그 흔적을 보는 것. AC에선 **완벽한 차단이 아니라 시그널 + 우회 가정**.

한눈에 (암기용)

#	기법	한 줄 원리	우회 한 줄
1	OS 디버그 플래그	PEB/API로 "디버거 붙음" 표시 조회	플래그·반환값 조작 / API 훅
2	타이밍 (`RDTSC`)	구간 사이클이 비정상적으로 김	임계값·패치, 멈추지 않고 분석, 오탐 유도

3	SW BP (INT3/0xCC)	코드 1바이트를 0xCC로 교체	HW BP로 전환, 해시 밖·복원 타이밍, 검사 무력화
4	HW BP (DR0–7)	레지스터에 주소≤4개, 실행/접근 시 CPU가 예외	DR 클리어·속임, 개수 제한 이용, 커널/다른 경로
+	코드 무결성	해시로 패치·훅 탐지	HW BP, 검사 루틴 무력화, 대상 밖 변조
+	VM (짧게)	CPUID/타이밍으로 분석환경	오탐 많음 → 점수화만

---

1) OS 디버그 플래그 (attach 탐지)

원리
Windows가 디버거가 붙으면 프로세스에 표시를 남김.

- `IsDebuggerPresent` / PEB `BeingDebugged`
- `NtQueryInformationProcess` (예: DebugPort 등) — Native API로 프로세스 정보 조회

탐지하는 것
디버거 **attach** 자체. BP를 안 걸어도 잡힐 수 있음.

우회 (개념)

- PEB 플래그를 꺼진 것처럼 보이게 패치
- `IsDebuggerPresent` / `NtQuery*` **훅**해 "디버거 없음" 반환
- 디버그 포트 관련 정보를 속이거나, 탐지 코드를 패치

오탐
거의 없음(의도적 디버깅이 아니면). 대신 **우회가 쉬움** → 단독 ban 부적합.

---

2) 타이밍 — RDTSC (Intel 연결 강함)

원리
RDTSC는 add/mov와 같은 **x86/x64 어셈블리 명령어**(Read Time-Stamp Counter). CPU 타임스탬프 카운터를 레지스터로 읽는다.
C/C++에서는 **인라인 어셈**(`__asm { rdtsc }` / `asm volatile("rdtsc"...)`)이나 컴파일러 intrinsic(`__rdtsc()` 등)으로 호출하는 경우가 많다.
짧은 코드 전후 차이가 크면 → 중간에 멈춤/싱글스텝 했을 가능성.

탐지하는 것
"붙어 있음"보다 **실행이 비정상적으로 느린 구간**(브레이크·스텝).

우회 (개념)

- 탐지 구간의 비교/임계값을 패치
- 해당 구간을 건너뛰게 분기 변조
- 가능한 한 멈추지 않는 분석(조건부·로그)
- (환경) VM/로드로 원래 느리면 탐지 측도 고통 → 임계값 싸움

오탐
절전, 스케줄링, 백그라운드 부하, VM/클라우드.
→ **단독 판정 금지**, 반복 측정·다른 시그널과 결합.

---

3) 소프트웨어 BP — `INT3` (`0xCC`)

원리
디버거가 명령 1바이트를 `0xCC`로 덮음. 원본은 **디버거 쪽 테이블**에 백업.
CPU가 `0xCC` 실행 → 브레이크 예외 → 디버거가 받음.

탐지하는 것
코드에 **남아 있는 `0xCC` 패치**(또는 체크썸 불일치).
※ 디버거만 붙고 BP 안 걸면 **이 기법만으로는** 안 잡힘.

우회 (개념)

- **하드웨어 BP**로 바꿔 코드를 안 고침
- 체크썸이 **일부 구간만**이면 그 **밖**에 SW BP
- 검사 직전 원본 복구 → 검사 후 다시 `0xCC`
- 체크썸/비교 루틴 자체 무력화
- (전체가 해시여도) SW BP 숨기기는 어렵고, 위 다른 우회로 감

오탐
드묾. 다만 패커/런타임 코드 생성과 혼동 주의.

---

4) 하드웨어 BP — DR0~DR7 (Intel 핵심)

원리
CPU 디버그 레지스터:

- DR0–DR3: 감시 주소 **최대 4개**
- DR7: 실행 / 쓰기 / 읽기·쓰기, 길이
- DR6: 어떤 조건으로 걸렸는지

`RIP`가 그 명령을 실행하거나, 데이터 BP면 해당 주소 접근 시 `#DB`.
메모리 바이트는 그대로 → `.text` 체크썸에 `0xCC` 안 보임.

탐지하는 것
스레드 컨텍스트의 DR이 비정상(주소·DR7 enable 등).
`GetThreadContext` 등으로 관찰(권한·환경에 따라 제품화 방식 다름).

우회 (개념)

- 탐지 전에 DR을 비운 것처럼 보이게 / 탐지 코드 패치
- 슬롯 4개 한계 → 꼭 필요한 곳만, 또는 다른 분석 수단
- 유저 탐지를 피해 더 아래 계층에서 관측·조작(개념만; 제품·서명·HVCI 제약)

오탐
정상 툴/다른 보안 제품이 DR을 쓰는 경우는 드물지만, **권한·호환** 이슈는 큼.

(+) 코드 무결성 (안티디버깅과 세트)

원리
코드/임포트 구간 해시·비교 → `0xCC`, 인라인 훅, 탐지 함수 패치 탐지.

우회 (개념)

- HW BP(코드 불변)
- 해시 **미포함** 영역·다른 모듈·동적 코드
- 검사 루틴/스레드 정지·비교 항상 성공으로 패치
- 전체가 해시여도 "디버깅 불가"는 아님 (위 우회들)

오탐
핫패치, 일부 보호/로더, 자기수정 코드(드묾).

(+) VM 탐지 (보너스 한 줄)

원리: `CPUID` 하이퍼바이저 비트·벤더, 타이밍 이상.
우회/한계: 일반 유저도 VBS/Hyper-V/클라우드 → **VM=치트 금지**, 리스크 점수만.

시험장 모범 답 (2–3분)

> 안티디버깅은 한 API가 아니라 (1) OS 디버그 플래그로 attach, (2) RDTSC로 정지/스텝 지연, (3) INT3 패치와 코드 무결성, (4) DR 기반 하드웨어 BP를 봅니다.
> 각각 플래그 조작·임계값/패치·HW BP 전환·DR 은닉 등으로 우회 가능하고 타이밍은 오탐이 커서, **다신호 + 서버**, 클라 단독 하드밴은 지양합니다.

꼬리 질문 빠른 답

질문	답
attach만으로 INT3 탐지?	아니요. INT3는 BP 패치용. attach는 플래그/NtQuery 쪽.
전체 .text 해시면 SW BP 숨기기?	패치가 남는 한 어려움. 그래도 HW BP·검사 무력화로 우회 여지.
IsDebuggerPresent만?	플래그 한 줄 우회 → 부족.

`플래그(attach) → RDTSC(지연) → 0xCC(패치) → DR(무패치 BP) → 무결성`
`전부 우회 가능 / 타이밍 오탐 / 다신호+서버`

1.2 `malloc`/`free` vs `new`/`delete`

⏱️ 직전1h · 33–40분(한 줄·표)

	C `malloc/free`	C++ `new/delete`
언어	C 라이브러리	C++ 연산자
생성자/소멸자	호출 안 함	`new`→생성자, `delete`→소멸자
타입	`void*` 반환, 캐스팅 필요	타입 안전
실패	보통 `nullptr`	기본은 `bad_alloc` (nothrow new 예외)
배열	`malloc(n*sizeof)`	`new[]` / `delete[]` 짝 필수
오버로드	불가(대체 가능은 별개)	클래스별 `operator new` 가능
섞어 쓰기	**미정의 동작(Undefined Behavior)** (`new`한 걸 `free` 등)	금지

AC/시스템에서 포인트

- Win32 힙/`HeapAlloc`와도 또 다름. CRT 힙과 Windows 힙을 혼동하지 말 것.
- 보안 모듈에서는 raw `new`보다 **RAII (`unique_ptr`, `vector`)** 가 수명·예외 안전에 유리.
- placement new, over-aligned allocation(C++17)까지 물으면 가산점.

한 줄

> `malloc`은 메모리만, `new`는 메모리 + 객체 수명(생성/소멸)을 책임진다. 짝을 맞추지 않으면 미정의 동작(Undefined Behavior).

1.3 `volatile`이란? (학부생 버전)

⏱️ 직전1h · 33–40분(한 줄·표)

한 줄

> 컴파일러에게: **"이 변수는 네가 생각하는 것보다 자주/이상하게 바뀔 수 있으니, 읽기·쓰기를 함부로 없애거나 합치지 마."**

왜 필요해?

컴파일러는 보통 코드를 빠르게 만들려고 최적화를 한다.
예를 들어 변수를 레지스터에만 두거나, "어차피 안 쓰이니까" 대입을 지울 수 있다.

그런데 어떤 변수는 **프로그램 밖**(하드웨어, 다른 장치)에서 값이 바뀌거나, **적어 둔 쓰기마다** 의미가 있을 수 있다.
그때 최적화가 있으면 동작이 깨진다 → `volatile`로 "이 변수만큼은 최적화로 건드리지 마"라고 표시한다.

쉬운 예

```
volatile int a = 0;
a = 0;
a = 1;
a = 3;
```

- **`volatile` 아님:** 중간 대입이 의미 없다고 보고 `a = 3;`만 남길 수 있음
- **`volatile` 임:** `a=0`, `a=1`, `a=3` **쓰기 세 번**이 그대로 남음 (중간 생략 방지)

읽는 경우도 비슷하다. 루프에서 `while (flag)` 할 때 `flag`가 다른 곳에서 바뀌는데 컴파일러가 "한 번만 읽고 끝"으로 최적화하면 무한루프가 될 수 있다. (`volatile`이 전통적으로 이런 이야기에 나오지만, **스레드끼리는 아래 주의**.)

하는 일 / 안 하는 일

하는 일	안 하는 일
그 변수 접근이 최적화로 **사라지거나 합쳐지는 것**을 줄임	스레드 안전 **보장 안 함**
"메모리에 실제로 읽고 써라"에 가깝게 동작	원자적 증가, 순서 보장 **아님**
	인라인 어셈 전체를 보호하는 용도 **아님**

스레드끼리 공유 플래그/카운터 → `std::atomic` 또는 mutex.
`volatile` ≠ 멀티스레드용.

어디서 쓰나 (감만)

- 하드웨어 레지스터처럼 **메모리 주소에 읽고 쓰는 게 곧 I/O**인 경우 (임베디드/드라이버)
- (시험) "스레드에 volatile 쓰면?" → **부족하다, atomic 써야 한다**

시험 한 줄

> `volatile`은 **해당 변수에 대한 최적화(생략·캐싱)를 억제**한다. 스레드 동기화가 아니다.

1.4 `const` vs `constexpr`

> ⏱️ 직전1h · 25–33분(표)

★ 시험 답변 (쉽게 · 이 정도면 OK)

`const`는 **값을 바꿀 수 없게 잠그는 것**이다.
한 번 넣어 두면 그 뒤에는 대입이 안 된다. 다만 그 값을 **언제** 정하는지는 실행 중이어도 된다. 예: `const int n = rand();`처럼 실행할 때 정해도, 정해진 뒤에는 못 바꾼다.

`constexpr`은 **계산식까지 포함해서, 가능하면 컴파일할 때 미리 계산해 상수로 굳히는 것**이다.
`constexpr int m = 3 + 4;`처럼 `3+4`를 컴파일러가 미리 7로 만든다.
그래서 `constexpr int bad = rand();`처럼 실행해 봐야 아는 값으로는 보통 못 만든다.

함수에도 붙일 수 있다.

- 멤버 함수 **뒤**의 `const` → "이 함수는 **객체를 안 바꾼다**"
- **앞**의 `constexpr` 함수 → "인자가 상수면 **미리 계산할 수 있다**"

한 줄: `const`는 나온 값에 자물쇠, `constexpr`은 계산까지 컴파일할 때 미리 끝내 버리는 상수.

이름으로 외우기

- **`const` = Constant (값 고정)**
  → "이미 나와 있는 값(Constant)에 **자물쇠**를 채운다." (언제 채워졌는지는 런타임이어도 OK)
- **`constexpr` = Constant + Expression (연산식 통째로 고정)**
  → "계산해야 하는 연산식(Expression)을 **컴파일러가 미리 계산**해서 상수로 굳힌다."

	`const`	`constexpr`
의미	초기화 후 **수정 불가**	**컴파일 타임에 값이 확정**될 수 있는 상수/함수
이름 감	Constant에 자물쇠	Constant + **Expression**을 미리 계산
초기화 시점	**런타임 가능** (`const int x = f();`). 오른쪽이 상수면 컴파일에 정해지기도 함. "항상 런타임 지연"은 아님	**컴파일 타임에 값이 정해져야** 함. `constexpr int bad = rand();` → 컴파일 에러

함수	const 멤버함수 = 객체를 논리적으로 안 바꿈	constexpr 함수 = 인자만 맞으면 컴파일 타임 평가 가능
배열 크기 등	항상은 아님	컴파일 타임 상수로 쓰기 좋음
에러 시점	잘못된 대입 등은 대개 **컴파일 에러**. (const_cast로 원본 const 수정 → 미정의 동작)	컴파일 상수여야 하는 자리에 못 쓰면 **컴파일 에러**

학부생 한 줄

- const = 값에 자물쇠 (**초기화는 런타임에도 OK**)
- constexpr = **식 전체를** 컴파일할 때 계산·확정 (런타임 rand()로 초기화 불가)
- constexpr 변수가 "런타임에 존재하지 않는다"는 뜻은 아님. **값이 컴파일에 확정**된다는 뜻에 가깝다.

```
const int n = rand();        // OK – 실행 때 정해짐, 이후 변경 불가 (자물쇠)
constexpr int m = 3 + 4;     // OK – 식 3+4를 컴파일 때 7로 굳힘
constexpr int bad = rand(); // 컴파일 에러 – 식을 미리 계산 못 함
int arr[m];                  // OK (m은 컴파일 상수)
```

constexpr 함수 (있음)

인자가 **컴파일 타임 상수**이면 호출 결과도 컴파일 때 계산될 수 있다.
런타임 인자를 넘기면 **일반 함수처럼 실행 때** 돌아간다 (둘 다 가능).

```
constexpr int square(int x) { return x * x; }  // C++11: 사실상 단일 return 등 제약

constexpr int a = square(3);  // 컴파일 때 9
int b = square(n);            // n이 런타임이면 실행 때 계산 (OK)
int arr2[square(4)];          // 배열 크기 등 "컴파일 상수" 자리에 사용 가능
```

	const 멤버 함수 foo() const	constexpr 함수
붙는 곳	멤버 함수 **뒤**	함수 선언 **앞**
의미	객체 **안 바꿈**	가능하면 **컴파일에 계산**
헷갈림	"불변 메서드"	"미리 계산 가능한 함수"

C++11 기준 말해야 할 것

- constexpr 변수: 리터럴 타입, 상수 표현식으로 초기화
- constexpr 함수: 사실상 단일 return·로컬 거의 불가 등 **제약 강함** (C++14에서 완화) — "11은 까다로웠다"고 말하면 가점

한 줄

> `const` = Constant에 자물쇠. `constexpr` = Expression(변수·**함수**)을 컴파일에 굳힐 수 있음.

1.4b ★ const 특집 (한곳에 모음)

> ⏱️ 직전1h · 25–33분(§6 move · §11 한 줄 위주)

`const` VS `constexpr` 차이 → 위 `### 1.4.`
여기선 **어디에 `const`가 붙는지**와 시험·코드리뷰에서 터지는 함정만 정리.

0. 한 줄 정의

`const` = **이 경로로는 바꾸지 마** (컴파일러가 대입·비const 호출을 막음).
위치(변수 / 포인터 / 멤버함수 / 매개변수)마다 **무엇이 불변인지**가 달라진다.

1. 변수 · 객체

```
const int n = 1;      // n 대입 불가. 초기화는 런타임 OK (1.4)
const Packet a(...); // a의 non-const 멤버함수 호출 불가
```

- 초기화 후에만 불변. **생성 중(생성자 본문)** 에는 아직 "완성 전"이라 멤버에 쓰는 것은 별개 규칙(아래 4·5).

2. 포인터 — * 왼쪽 vs 오른쪽 (단골)

```
const int* p;        // 가리키는 int는 불변 (p는 다른 주소로 옮길 수 있음)
int* const p;        // 포인터 자체 불변 (가리키는 int는 수정 가능)
const int* const p;  // 둘 다
```

선언	주소(p)	가리키는 값(*p)
`const int* p`	바꿀 수 있음	**못 바꿈**
`int* const p`	**고정**	바꿀 수 있음
`const int* const p`	**고정**	**못 바꿈**

```
int a = 1, b = 2;

const int* p = &a;
//*p = 10;   // 불가 — 값에 자물쇠
p = &b;      // OK — 다른 주소로 옮기기 가능

int* const q = &a;
```

```
*q = 10;      // OK — 값은 수정 가능
//q = &b;     // 불가 — 주소(포인터)에 자물쇠
```

읽는 팁: **`const`가 `*`보다 왼쪽 → 데이터(값) 잠금 / 오른쪽 → 포인터(주소) 잠금.**
`const int*` ≈ `int const*` (같은 뜻).
헷갈림 주의: `const int* p`는 **주소를 고정하는 게 아님** (그건 `int* const`).

3. 참조 · 함수 인자

```
void f(const int& x);   // 읽기 전용, 복사 없음
void g(int& x);         // 수정 가능 out-파라미터
void h(int x);          // 복사본. 소유·독립 값
```

- 임시(rvalue)는 **`const int&`에는 붙음, `int&`에는 안 붙음.**
  (큰 객체면 같은 패턴으로 `const std::string&` 등.)

4. const 멤버 함수 (`void foo() const`)

```
struct S {
  int x;
  mutable int cache;           // const 메서드에서도 수정 가능 (캐시·mutex 등)
  int get() const { return x; } // this 타입이 const S*
  void set(int v) { x = v; }    // non-const만
  int&       at(size_t i);
  const int& at(size_t i) const; // const 오버로드 — 객체 const성에 따라 선택
};
```

포인트	내용
`this`	const 메서드 안 `this` = `const S*` → 멤버 대입 기본 불가
호출	**const 객체**에서는 const 메서드만 호출 가능
비트 vs 논리	표준은 "멤버를 수정하지 않음"(bitwise에 가깝). `mutable`로 논리 const 허용
포인터 멤버	`int* p`를 const 메서드에서 `*p = ...` 하는 건 **포인터 자체는 안 바꾸므로 컴파일 OK** — 논리 const 깨지기 쉬움 → 리뷰 포인트
오버로드	`at` / `operator[]`처럼 const / non-const 쌍이 흔함

5. 생성자와 const

- **생성자·소멸자에는 `const`를 붙일 수 없음** (`S() const` 불가). 객체를 "만드는/부수는" 단계라 const 메서드와 성격이 다름.
- **`const` / 참조 멤버**는 생성자 **멤버 이니셜라이저**로만 초기화 가능 (본문 대입 불가).

```
struct S {
  const int id;
  int& ref;
  S(int i, int& r) : id(i), ref(r) {}  // 필수
};
```

- **const 멤버가 있으면** 컴파일러 생성 **이동**이 삭제·비활성되거나 사실상 복사로 떨어지기 쉬움 → `std::move`가 기대한 만큼 안 먹을 수 있음 (클래스 리뷰: “move 막힘”).

6. 우측값 · move와 const (★★ 함정)

```
const std::string s = "hi";
std::string y = std::move(s);  // 대개 **복사**. 이동 아님
```

1. `move(s)` → `const string&&` 로 캐스트는 됨
2. move 생성자 `string(string&&)`는 **non-const rvalue**만 받음
3. `const string&&` → 복사 생성자 `string(const string&)`

한 줄: const + move ≈ copy. (`string(const string&&)` 같은 생성자는 실무에 거의 없음.)
상세 연습 → `### 1.6b` M2.

바인딩	가능?
`int&&` ← 일반 rvalue (예: `1`, `std::move(n)`)	OK
`int&&` ← `const int` / `const int&&`	불가 (const 벗겨지지 않음)
`const int&` ← lvalue / rvalue	OK
`const int&&`	거의 안 씀. move 훔치기 불가

7. const_cast · 미정의 동작

```
const int a = 1;
int& r = const_cast<int&>(a);
r = 2;   // 원본이 const 객체 → **미정의 동작(Undefined Behavior)**
```

- `const_cast`는 “원래 non-const였는데 const 참조로만 받은 경우” 되돌릴 때.
- **진짜 const 객체 수정 = UB.** → E06 / 캐스팅 4종.

8. auto와 const

```
const int cx = 1;
auto a = cx;        // a 타입 = int  (top-level const 벗겨질 수 있음)
const auto b = cx;  // const int
auto& c = cx;       // const int&
```

필요하면 `const auto&`를 명시.

9. 반환값에 const

```
const int f();              // 값 반환에 const – 거의 이득 없음
const int& g() const;       // 멤버 int 노출·수명 주의 (댕글링)
int* const h();             // "포인터 값"이 const – 의미 약한 경우 많음
```

리뷰: **값 반환 `const int`는 지양**, 참조/포인터 반환은 수명·const 정확성 확인.

10. 코드리뷰에서 const 체크리스트

1. 읽기만 하는 인자 → `const int&` / `const string&` (또는 view) 빠진 곳
2. 논리적으로 불변인 메서드 → `const` 누락 / 반대로 const인데 상태 변경
3. `const` 멤버·`mutable` 남용
4. `const` 객체/`const` 멤버 때문에 **move가 복사로 바뀌는지**
5. 포인터 const 방향 (`const int*` vs `int* const`) 의도
6. `const_cast`로 원본 const 수정 → UB
7. const / ref 멤버 → 이니셜라이저·복사/이동 특수 멤버 영향

11. 면접 한 줄 모음

- "`const`는 경로상 불변, `constexpr`은 컴파일에 값이 확정."
- "멤버 `foo() const`는 `this`가 `const`라 객체 안 바꿈. 캐시는 `mutable`."
- "생성자에 `const` 붙일 수 없고, `const`/ref 멤버는 이니셜라이저 필수."
- "`const`에 `move` 하면 보통 복사다."
- "`const int*`는 데이터 보호, `int* const`는 포인터 주소 고정."

1.5 `std::move` / `std::forward`

⏱ 직전1h · 5–25분 ★본진

`std::move`

- 실제 이동을 **수행하지 않음**. **rvalue 참조로 캐스팅**할 뿐.
- 이동 생성자/이동 대입이 호출될 **자격**을 줌.
- 이동 후 원본은 **valid but unspecified** 상태 → 대입·소멸 외 사용 주의.

`std::forward`

- **완벽 전달(perfect forwarding)** 용.
- 템플릿에서 `T&&` (포워딩 참조)로 받은 인자의 **값 범주(lvalue/rvalue)** 를 유지한 채 넘김.

- `std::forward<T>(x)` — `T`가 어떻게 추론됐는지가 핵심.

언제 무엇

상황	사용
이름 있는 객체를 이동시켜 넘김	`std::move`
`T&&`로 받은 인자를 그대로 전달	`std::forward<T>`
이미 rvalue인 임시	보통 그냥 넘김 / 필요 시 move

자소서와 연결 (본인 경험)

> CDN에서 받은 복호화 버퍼·모듈 핸들을 `unique_ptr` 등으로 소유하고, 하위로는 `move`/참조만 전달해 복사·이중 해제를 줄였다.

함정 (요약)

- const 객체는 move가 복사로 fallback
- 이름 있는 `T&&` 변수는 식으로는 **lvalue**
- 자기 자신을 move로 채우는 실수
- 컨테이너에서 move 후 iterator/참조 무효화 규칙
- 로컬 `return std::move(x)` → RVO 방해 가능 (아래 `1.5b`)

1.5b ★ RVO / NRVO (반환값 최적화)

> ⏱ 직전1h · 5–25분

RVO = Return Value Optimization.
함수가 **값으로 반환**할 때, 임시/로컬 객체를 호출부 결과 자리에 **처음부터 직접 생성**해 **복사·이동을 생략**하는 최적화(copy elision).

이름	무엇
RVO	`return T(...);` 처럼 **임시(prvalue)** 를 반환할 때 생략
NRVO	Named RVO. `T local; ...; return local;` 처럼 **이름 있는 로컬** 반환 시 생략(컴파일러·경로에 따라)

```
T make() {
  return T(1, 2);   // RVO 후보 — 호출측 자리에 바로 생성될 수 있음
}

T make2() {
  T local(1, 2);
  return local;     // NRVO 후보 — 가능하면 복사/이동 없음
}
```

```
T a = make();       // 이상적: 생성 1번. (이동/복사 생성자 호출 안 볼 수도)
```

C++11 vs C++17 (면접용 한 줄)

C++11/14	RVO/NRVO는 **허용(as-if)**. 대부분 컴파일러가 함. 의무는 아님.
C++17	**prvalue**를 같은 타입으로 초기화하는 경우 등 **일부 copy elision 필수**. NRVO는 여전히 선택.
Pre-Test 답	"반환은 값으로 두고 컴파일러 RVO에 맡긴다. 로컬에 `return std::move` 하지 않는다."

★ 함정: `return std::move(local)` (실무에서 제일 중요)

실무는 `return T(...);`(순수 RVO)보다 `T local; …; return local;`(NRVO 후보)가 더 흔하다.
그래서 **경각심의 핵심 = 이름 있는 로컬에 생각 없이 `move` 하지 말 것.**

```
std::string make_name() {
  std::string local = "krafton";
  local += "-kss";
  return local;               // OK — NRVO 후보. 안 돼도 컴파일러가 move 취급하기도 함
  // return std::move(local); // 나쁨 — NRVO 깨고 이동 강제
}
```

(`int` 같은 스칼라는 비용이 거의 없어 RVO 논의 대상이 아님 → `string`/`vector` 등으로 설명.)

★ 로컬은 `return`만 / 파라미터·멤버는 `move` 후보

같은 `return` + `move`라도 **무엇을 반환하느냐**가 갈린다.

반환 대상	코드	`move`?	이유
함수 안 **로컬**	`return local;`	**쓰지 마**	곧 죽음 → NRVO/자동 처리. `move`는 최적화 방해
값으로 받은 **파라미터**	`return std::move(other);`	**쓰는 편**	NRVO 대상 아님. 그냥 return이면 복사 날 수 있음
멤버	`return std::move(buf);`	**쓰는 편**	함수 끝나도 객체에 남음 → 자동 "훔치기" 안 함

① 로컬 — move 금지(도와준다고 쓰지 마)

```
std::string make_name() {
  std::string local = "krafton";
  local += "-kss";
```

```
  return local;              // 정석
}
```

② 매개변수 — move가 필요할 수 있음

```
std::string wrap(std::string other) {  // 값으로 받음 → 이미 우리 소유
  other += "!";
  return std::move(other);   // 소유 이전. return other 는 복사로 떨어질 수 있음
}
```

파라미터는 "이 함수가 지은 로컬"이 아니라 **호출자가 넘긴 이름**이라 NRVO 대상이 아님.
(최신 툴체인은 암시 move에 더 관대할 수 있으나, 면접·리뷰 구분: **파라미터=move, 로컬=그냥 return**.)

③ 멤버 — move가 필요할 수 있음

```
struct Holder {
  std::string buf;
  std::string take() {
    return std::move(buf);   // 멤버를 비우며 넘김
    // return buf;           // 보통 복사. buf는 함수 종료로 안 사라짐
  }
};
```

직관

- 로컬 = "이 줄에서 수명 끝" → 컴파일러에 맡김 (`return local`)
- 파라미터/멤버 = "아직 안 죽을 수도 있는 이름" → 명시적으로 훔침 (`return std::move(...)`)

move와의 관계

- RVO/NRVO가 터지면 **이동 생성자도 안 불림** (아예 생략).
- "이동이 있으니 복사는 걱정 없다"만으로 끝내지 말고, **반환은 elision이 1순위**라고 말하면 좋음.
- 이동 생성자가 `= delete`여도, **필수 elision(C++17 prvalue)** 경로면 반환이 가능한 경우가 있음 (세부 면접용).

언제 RVO/NRVO가 어려워지나 (감만)

- 분기마다 **다른 이름**의 로컬을 return
- 조건에 따라 T / 다른 타입 혼재
- 추상·다중 반환 경로로 NRVO 포기

그래도 **로컬에 `return std::move(local)`로 "도와준다"고 쓰지 말 것.**

면접·리뷰 한 줄

- “값 반환 + RVO/NRVO. **로컬은 `return local`, `move(local)` 금지.**”
- “파라미터·멤버 소유 이전만 `return std::move(...)`.”
- “실무 경각심의 중심은 NRVO — 로컬에 생각 없이 move 하지 말 것.”
- 복사 줄이기 세트: `const&` / move / **RVO** / 불필요 값 전달 금지 (C07).

1.6 참조 규칙: `&` / `&&` / `& &&` / `&& &` (참조 축소)

⏱ 직전1h · 5–25분 ★본진

기본

- `T&` : lvalue 참조
- `T&&` : rvalue 참조 (단, 템플릿 `T&&`는 포워딩 참조일 수 있음)

참조 축소 (reference collapsing) — `typedef`/`using`/템플릿 인자 조합 시

조합	결과
`&` + `&`	`&`
`&` + `&&`	`&`
`&&` + `&`	`&`
`&&` + `&&`	`&&`

암기: 하나라도 lvalue 참조(`&`)가 있으면 결과는 `&`. 둘 다 `&&`일 때만 `&&`.

포워딩 참조

- `template<class T> void f(T&& x)` 형태에서 `T`가 추론될 때
- lvalue 인자 → `T = U&` → `U& &&` → `U&`
- rvalue 인자 → `T = U` → `U&&`

이 규칙 때문에 `std::forward`가 동작한다.

1.6b move/forward 함정 상세 (연습 M1~M3)

⏱ 직전1h · 5–25분(암기 치트·표만, 재풀이 X)

타입(선언)과 **값 범주(식)** 를 섞지 말 것. 여기 세 줄이 중~중상 단골.

0. 두 층을 분리

층	뜻	예

타입	변수가 선언된 타입	`std::string&& s`
값 범주	그 **식**이 lvalue인지 rvalue인지	식 `s`, 식 `std::move(s)`

암기: 이름이 있는 식은 (거의 항상) **lvalue**. 타입에 `&&`가 있어도 예외 아님.

M1 — 이름 있는 rvalue 참조는 lvalue

```
void f(std::string&& s) {
    std::string a = s;             // copy  ← 식 s는 lvalue
    std::string b = std::move(s);  // move ← 다시 rvalue로 캐스트
}
f(std::string("hi"));  // 인자로 넘길 때는 우측값만 바인딩
```

- 매개변수 타입 `string&&` = "우측값만 **받을** 수 있음"
- 함수 안 식 `s` = 이름 있음 → **lvalue** → 복사 생성자 선택
- 다시 옮기려면 명시적 `std::move(s)`
- `std::move`는 이동 수행이 아니라 **rvalue 참조 캐스트**

왜 이렇게? 들어온 뒤에도 항상 rvalue면 실수로 두 번 move하기 쉬움 → 언어가 "받은 뒤엔 이름 =lvalue"로 고정.

M2 — `const`면 move가 사실상 안 먹힘

```
const std::string s = "hi";
std::string t = std::move(s);  // copy
```

1. `std::move(s)` → 대략 `const std::string&&` (캐스트는 됨)
2. move 생성자 `string(string&&)`는 **non-const** 우측값만 받음
3. `const string&&`는 거기에 못 붙음 → `string(const string&)` **copy**

한 줄: `move` **함수가 막히는 게 아니라, "훔쳐 갈 생성자"가 const에 매칭이 안 된다.**
일반 `string`/`vector`/`unique_ptr` 기준 → **const + move = copy** 로 답하면 됨.
(`T(const T&&)` 같은 이상한 생성자는 거의 없음.)

M3 — 포워딩 참조의 T 추론 (T ≠ x의 타입)

★★★Forwarding Reference에서 lvalue → T=U&, rvalue → T=U(T랑 x 맞추는 문제)★★★

```
template<class T>
void f(T&& x);   // T가 추론될 때만 포워딩 참조

int a = 1;
const int b = 2;
f(a);  // T = int&       , x 타입 = int&
```

```
f(b);  // T = const int& , x 타입 = const int&
f(3);  // T = int        , x 타입 = int&&   ← T는 int지 int&&가 아님!
```

인자	T (추론 결과)	x의 타입 (T&& 축소 후)
lvalue `int a`	`int&`	`int&`
lvalue `const int b`	`const int&`	`const int&`
rvalue `3`	**`int`**	**`int&&`**
rvalue `std::move(a)`	**`int`**	**`int&&`**

- lvalue → "좌측값 유지"를 위해 T에 참조가 들어감 → T&& 축소로 U&
- rvalue → T는 값 타입 U → 매개변수 T&& = U&&

헷갈림 포인트: 우측값으로 불렀을 때 "&&니까 T도 int&&"가 아님.
T는 int, **x가 int&&.**

포워딩 참조 vs 그냥 rvalue 참조

```
void g(std::string&& x);     // 구체타입&& → 우측값만. 포워딩 아님
template<class T> void h(T&& x); // 추론 T&& → 좌/우 둘 다
```

M4 — 같은 인자를 forward로 두 번 (연습 Q7)

```
template<class T>
void twice(T&& x) {
    sink(std::forward<T>(x));  // 1번째
    sink(std::forward<T>(x));  // 2번째 ← 위험할 수 있음
}
twice(std::string("a"));  // T = string → forward가 둘 다 move
```

원리: 우측값으로 넘기면 첫 sink가 자원을 **move로 가져감**.
둘째 forward도 다시 우측값으로 취급 → **이미 moved-from**인 객체를 또 넘김 (비었거나 valid but unspecified).

한 줄: 완벽 전달은 "한 번"이 기본. 같은 rvalue를 forward로 두 번 move하지 말 것.
(좌측값으로 twice(s)면 T = string&라 forward가 참조만 넘기므로 두 번이 덜 치명적일 수 있으나, sink가 수정하면 다른 문제.)

초짧은 셀프체크 (정답)

```
void f(string&& s) { string x = s; }     → copy
const string c; string y = move(c);      → copy
template<class T> void g(T&&); g(n); g(1); → T=int& / T=int
twice(string("a"))에서 forward 두 번 → 두 번째 moved-from
```

암기 치트시트

```
① 이름 있는 && 변수 = lvalue  → 다시 move 필요
② const + move = 보통 copy   (move ctor가 const 거절)
③ f(lvalue)→ T=U&  /  f(rvalue)→ T=U   (x는 U&&, T가 U&&가 아님!)
④ forward 두 번(rvalue) = 두 번째 moved-from
타입(선언) ≠ 값 범주(식)
T(추론) ≠ 항상 x의 최종 타입
```

1.7 함수 호출 규약 (Calling Convention)

⏱️ 직전1h · 54–58분(표)

x86에서는 cdecl은 스택을 caller가 정리하며 주로 C 라이브러리 계열이 사용하고, stdcall은 callee가 스택을 정리하며 WinAPI에서 사용됨. x64는 파라미터를 레지스터(rcx, rdx, r8, r9)를 통해 전달하며 스택은 caller가 정리한다.

설명해야 할 항목

1. 인자를 **누가, 어떤 순서**로 스택/레지스터에 올리는가
2. **누가 스택을 정리**하는가 (caller vs callee)
3. **반환값**은 어디에
4. **이름 맹글링 / 장식** 차이 (특히 Windows)

규약	대략	비고
cdecl	인자 오른쪽→왼쪽 스택, caller cleanup	C 기본(x86)
stdcall	callee cleanup	Win32 API 다수 (x86)
fastcall	일부 레지스터 전달	구현 의존
thiscall	this를 ECX 등	멤버 함수 (x86 MSVC)
x64 Windows	RCX, RDX, R8, R9 + 스택, caller가 shadow space 32바이트	사실상 단일 ABI

AC/리버싱 연결

- 후킹 시 규약을 모르면 인자/반환을 깨먹음
- x64는 규약이 단순하지만 **shadow home space**, 정렬, XMM 인자도 알아야 함
- 게임/치트 후킹 문제와 바로 연결됨

한 줄

> 호출 규약은 인자 전달·스택 정리·반환 위치의 계약이다. x86은 종류가 많고, Windows x64는 레지스터 기반 단일 ABI + shadow space.

1.8 특정 함수를 호출하는 게임 핵을 막는 법

> ⏱️ 직전1h · 54–58분(우회·오탐·다계층)

문제 의도: “후킹만 하면 된다”가 아니라 **공격면·우회·오탐**을 말하는지.

방어/탐지 아이디어 (계층)

1. **해당 API/함수 경로 관측**
   - IAT/EAT/인라인 훅 탐지, 직접 시스템콜 우회 가능성 언급
2. **호출 주체·컨텍스트 검증**
   - 정상 모듈 코드 범위에서 온 호출인지 (리턴 주소, 스택 워크, 모듈 화이트리스트)
3. **행위·결과 기반**
   - 함수 자체보다 “그 함수로 달성하려는 효과”(메모리 읽기, 입력 주입, 패킷)를 다신호로
4. **무결성**
   - 게임/AC 코드·임포트 테이블 변조 탐지
5. **서버 권한**
   - 클라에서만 막으면 우회됨 → 중요 판정은 서버

답변에서 꼭 넣을 말

> 단일 API 차단은 rename/직접 syscall/수동 매핑으로 우회되기 쉽다. 관측 지점을 분산하고, 오탐·성능을 전제로 정책화한다.

포트폴리오 연결

- 매크로: `PrintWindow` 단일 훅의 한계 → 다신호 관측으로 확장 (PPT 04-B)
- 행위분석: 후킹을 “차단”이 아니라 “관측 설계”로 쓴 경험

1.9 `CreateFile` / `ReadFile` 예제 코드 리뷰

> ⏱️ 직전1h · 40–48분

렌즈: 보안 / 에러처리 / 수명 / 레이스 / 성능

자주 나오는 지적 포인트 체크리스트:

#	찾을 것	왜 위험/이상한가
1	반환값/`INVALID_HANDLE_VALUE` 미검사 (`NULL`과 혼동)	이후 ReadFile 미정의 동작(Undefined Behavior)/크래시. **CreateFile 실패는 INVALID** (자세한 건

		1.11)
2	`GetLastError`를 실패 전에 호출	의미 없는 에러코드
3	버퍼 크기 vs `nNumberOfBytesToRead`	오버플로
4	`ReadFile`의 실제 읽은 바이트 (`lpNumberOfBytesRead`) 무시	부분 읽기
5	핸들 `CloseHandle` 누락 / 이중 닫기	누수·핸들 재사용 버그
6	경로에 사용자 입력 + 상대경로	path traversal
7	`GENERIC_ALL` 등 과도한 권한	최소 권한 위반
8	공유 모드 0으로 독점	호환성/데드락성 이슈
9	큰 파일을 한 번에 스택 버퍼로	스택 오버플로·성능
10	TOCTOU (존재 확인 후 다른 파일로 교체)	보안
11	ASCII/`char` 경로만 사용	Unicode 경로 실패
12	예외 경로에서 핸들 미해제	RAII 부재
13	성공해도 버퍼 미널 종료인데 문자열로 사용	C 문자열 버그
14	동기 I/O를 UI/게임 스레드에서	프레임 스파이크

답 구조

> (1) 버그/보안 이슈 나열 → (2) 각각 영향 → (3) 고치는 방향(RAII 핸들, 크기 검사, 부분 읽기 루프, 최소 권한)

1.10 클래스 선언 예제 — 위험 / 성능

> ⏱ 직전1h · 40–48분

체크리스트:

영역	찾을 것
Rule of 0/3/5	소멸자만 있고 복사/이동 없음 → 더블 free / 복사 비용
복사 비용	큰 `std::vector`/`string` 멤버를 값으로 전달, 불필요한 복사
가상 소멸자	기저 포인터로 delete하는데 non-virtual dtor
멤버 초기화	생성자 본문 대입 vs 멤버 이니셜라이저 / 초기화 순서
const 정확성	논리적으로 const인 메서드가 non-const

인라인/헤더	무거운 코드를 헤더에 → 컴파일·I-cache
false sharing / mutable	멀티스레드 공유 카운터
예외 안전	raw pointer 소유, new 도중 throw
인터페이스	구현 유출, pImpl 부재로 재컴파일 폭주
explicit	단일 인자 생성자 암시 변환
static 초기화	fiasco, 멀티스레드
가상함수 + 값 슬라이싱	값으로 받음
move가 막힘	const 멤버, 사용자 정의 복사만 존재

성능 쪽 모범 답

- const 참조/`string_view`로 받기
- move 가능하게 설계
- 작은 타입은 값, 큰 타입은 참조
- 가상호출이 핫패스면 CRTP/최종 클래스 검토 (과하지 않게)

1.11 ★ Pre-Test 보강 특집 (기출 옆에 붙일 개념)

⏱ 직전1h · 48–54분(한 줄 모음)

지난 10문에 직접은 없어도 **코드리뷰·C++ 함정**으로 잘 붙는 축.
우선 **1~5**를 말로 설명 가능하게, **6~10**은 한 줄·표만.

1. 정수·크기 함정 (`size_t` / 부호 / 오버플로)

```
int n = ReadSize();           // 음수·큰 값 가능
char* buf = new char[n];      // n<0 → size_t로 변환되며 거대 할당 / UB 위험
if (user_len + 1 < cap)       // user_len+1 이 size_t면 오버플로 후 작은 수 → 검사 통과
  memcpy(dst, src, user_len);
```

함정	왜
`int` 길이 VS `size_t`/`DWORD`	부호·폭 다름. 비교·할당 시 변환
`a + b < cap`	덧셈이 먼저 래핑되면 검사 무력화 → `a < cap && b <= cap - a`
`read > 0`만 보고 인덱스	`DWORD`/`size_t`와 `int` 섞기
signed overflow	**미정의 동작(Undefined Behavior)** (unsigned wrap은 정의됨)

리뷰 한 줄: **길이·인덱스·할당 크기는 부호·폭·오버플로를 같이 본다.**

2. C 문자열 · 위험 API · 널 종료

```
char buf[16];
strcpy(buf, src);     // src 길이 미검사 → OOB
sprintf(buf, "%s", s); // 동일
ReadFile(..., buf, 16, &n, ...);
puts(buf);            // n==16이면 널 없음 → 오버리드. buf[n]='\0' 필요(공간 17 또는 읽기 15)
```

피하기	대신
gets / strcpy / sprintf / strcat	strncpy_s/strcpy_s, snprintf, std::string
바이너리를 C문자열로 가정	길이(n)와 널을 **분리**해서 다룸

리뷰: **용량, 실제 길이, 널 자리(+1), 부분 읽기** 네 가지.

3. 정렬·패딩·#pragma pack / 구조체 레이아웃

```
struct P { char c; int x; };   // 패딩 생김. sizeof != 1+4 인 경우가 보통
#pragma pack(push, 1)
struct Wire { char c; int x; }; // 패킹 – 파일/네트워크 레이아웃용
#pragma pack(pop)
```

- 컴파일러는 멤버 **정렬** 위해 패딩을 넣음 → sizeof, offset이 “손계산”과 다를 수 있음.
- 패킷/파일/ReadFile로 raw 캐스팅할 때 pack·엔디안·정렬 깨진 접근(UB/버스 오류 가능) 주의.
- AC 감: 공유메모리·치트 구조체·맵 오프셋 리뷰 시 **레이아웃 가정**을 의심.

4. 가상함수 · vtable · 가상 소멸자 (한 장)

```
Base* p = new Derived;
p->foo();     // virtual이면 Derived::foo (vtable)
delete p;     // Base 소멸자가 non-virtual → **UB**/파생 소멸 스킵
```

용어	한 줄
vtable	클래스별 가상함수 주소 표
vptr	객체 안에 그 표를 가리키는 포인터(보통)
가상 소멸	기반 포인터로 delete할 때 **필수**
슬라이싱	Base b = derived; 값 복사 → 파생 부분 잘림

생성/소멸 **중** 가상 호출 → 기대와 다른 바인딩 가능(파생 미완성).

5. explicit / 변환 생성자

```
struct Buf {
  explicit Buf(int n);     // explicit 없으면 Buf b = 10; / f(10) 암시 변환
};
void f(Buf);
// f(10);  // explicit면 컴파일 에러 — 의도치 않은 할당/변환 방지
```

단일 인자 생성자·변환 연산자는 **`explicit` 기본 습관**. API가 조용히 열리면 리뷰 지적.

6. iterator / `vector` 무효화

```
for (auto it = v.begin(); it != v.end(); ++it) {
  if (need) v.push_back(x);  // 재할당 시 it·참조·포인터 **무효화** → UB
}
```

연산	`vector`
`push_back` (용량 초과)	전체 무효화 가능
`insert`/`erase`	해당 지점 이후 무효화
`reserve`로 여유	재할당 줄일 수 있음

콜백/훅 안에서 같은 컨테이너 수정 = 단골 버그.

7. `atomic` vs `volatile` (표로 끝)

	`volatile`	`std::atomic`
목적	컴파일러가 접근 **생략·합치기** 억제	스레드 간 **동기화·원자성**
data race	막지 않음	올바른 사용 시 정의된 동시 접근
AC/게임	MMIO 감	플래그·카운터·시퀀스

한 줄: 스레드 공유 → `atomic`(또는 mutex). `volatile`로 대체 **불가**. (상세 `### 1.3`)

8. HANDLE: `NULL` vs `INVALID_HANDLE_VALUE`

API	실패 반환	검사
`CreateFile`	`INVALID_HANDLE_VALUE` `((HANDLE)-1)`	`== INVALID_HANDLE_VALUE`
`CreateThread` / 많은 커널 객체	`NULL`	`== NULL`
`CloseHandle`	실패해도 대강 넘기지 말 것	이중 닫기 주의

```
HANDLE h = CreateFileW(...);
if (h == INVALID_HANDLE_VALUE) { /* GetLastError */ }
// if (!h)  ← CreateFile에선 잘못된 검사
```

9. SEH vs C++ 예외 (개념만)

	C++ 예외 (`try`/`catch`)	SEH (`__try`/`__except`)
성격	언어·타입 기반	Windows OS 구조적 예외 (접근위반 등)
컴파일러	표준	MS 확장
AC 감	리소스 해제=RAII가 본류	크래시·디버거·하드웨어 예외와 인접

섞어 쓸 때 규칙·해제 순서 이슈 있음 → Pre-Test는 **"다르다 / 크래시는 SEH 쪽"** 정도면 충분.

10. 페이지 권한 감 (`R` / `W` / `X`)

- 메모리 페이지에 읽기·쓰기·**실행** 권한이 따로 있음.
- **W+X 동시**(쓰기 가능한 실행 코드)는 훅·코드 주입·자기수정에 유리 → 보안상 싫어함 (DEP/NX, 가능한 한 분리).
- AC 한 줄: "코드 구역은 실행·무결성, 데이터는 쓰기 — **권한 경계**를 전제로 훅/패치를 본다." (`VirtualProtect`로 잠시 바꾸는 패턴이 있다는 **감**만, API 나열 X.)

보강 특집 — 면접·리뷰 한 줄 모음

1. 길이 검사 전에 **덧셈 오버플로·부호 변환** 봤는가.
2. C문자열은 **용량 / 길이 / 널 / 부분I/O**.
3. raw 구조체는 **패딩·pack·정렬**.
4. `Base*` delete → **가상 소멸**.
5. 단일 인자 생성자 → **`explicit`**.
6. 루프 중 `push_back` → **iterator 무효화**.
7. 스레드 → **`atomic` ≠ `volatile`**.
8. `CreateFile` 실패 → **`INVALID_HANDLE_VALUE`**.
9. 크래시급 → **SEH**, 일반 에러 흐름 → C++예외/에러코드.
10. 코드에 쓰기 권한 = **위험 신호(W⊕X)**.

1.12 ★ 개편 대비 소재 풀 (한 번 훑기)

⏱ 직전1h · 54–58분(**§0 뼈대만**)

용도: 기출 문장이 바뀌어도 나올 수 있는 **소재 지도**.
읽는 법: 수 D-1에 **30~40분 통독**. 각 칸 = "나오면 이 렌즈/한 줄". **암기·심화 금지.**
고정 축: ① C++11 정확성·UB·소유권 ② 코드리뷰 렌즈(보안/정확성/성능/Win) ③ 보안 답 뼈대(아래).

0. 소재가 바뀌어도 쓰는 답 뼈대

C++ / 리뷰: 정의 → 차이 → 함정(UB) → 고치는 방향(RAII·크기·수명).
보안·AC: 원리 → 무엇을 탐지/방지 → **우회면** → **오탐·성능** → (가능하면) 다계층·서버.
예: 안티디버깅이 **후킹 방지**로 바뀌어도 뼈대는 동일 (`1.8`과 같은 톤).

A. C++ · 코드 (기출 옆에 끼워 넣기 쉬움)

소재	나오면 뭐라고 / 어디
스마트포인터 오용	순환=`weak`, `get`≠소유, `release` 주의 → B0/B2
스레드 리뷰	data race, `join` 전 공유, deadlock, **volatile≠atomic** → `1.3`·`1.11`
컨테이너	iterator 무효화, 재할당 → `1.11` §6
예외·RAII	throw 경로 누수·핸들 → RAII/`unique_ptr`
레이아웃	패딩·pack·가상소멸·슬라이싱 → `1.11` §3–4
API 설계	`explicit`, raw 소유 포인터 → `1.11` §5, `1.10`
성능 리뷰	핫패스 할당·복사·동기 I/O → `1.10`·PART D
C API 혼용	`new`/`malloc` 짝, `char*` 수명·널 → `1.2`·`1.11` §2
move/forward/RVO	**개편돼도 잔존 가능성 높음** → `1.5`–`1.5b`–`1.6b`

B. Windows · 유저모드 (CreateFile 대체·확장)

소재	한 줄 감
다른 I/O	`WriteFile`/오버랩/`MapViewOfFile`/레지스트리 — **반환값·부분 I/O·수명** 동일 렌즈
프로세스/스레드	핸들 닫기·상속·대기. 생성 실패 검사
DLL / `DllMain`	loader lock — **LoadLibrary·동기화·COM 금지**에 가깝게
동기화 객체	Event/Mutex/CS 수명, 잘못된 wait
IPC	파이프(스트림·복사) vs 공유메모리(매핑·별도 동기화) → `### 3.2b`
`VirtualAlloc`	commit/reserve, **RWX** → `1.11` §10
Unicode	`A`/`W`, 길이=문자 vs 바이트
호출규약	콜백 캐스트 mismatch → `1.7`
HANDLE	`NULL` VS `INVALID_HANDLE_VALUE` → `1.11` §8

C. 보안·AC 사고 (안티디버그 자리 대체 후보)

소재	한 줄 + 뼈대 적용
후킹 방지·탐지	IAT/인라인/VEH **감**. 무결성·비교. 우회·오탐 → `1.8` 확장

코드·모듈 무결성	체크섬/해시, 페이지 권한, 자기수정
인젝션 관측	원격 스레드·매핑 DLL — **구현 X, 관측 포인트**
메모리 보호	치트 R/W vs 권한 분리 (R/W/X)
패킷·신뢰	클라 신뢰 금지, 서버 권위, 리플레이
입력·매크로	합성 입력 vs 실입력, 타이밍·오탐 (포트폴리오 연결)
분석 환경	샌드박스/VM 흔적 — 목록 암기 X, 시그널·오탐
안티탬퍼·크래시	실패 시 대응, DoS·오탐 트레이드오프
텔레메트리	무엇을 모으나, 민감정보·성능
우회 가정 설계	한방 방어 X → 다신호

후킹 방지 구술 예시 (뼈대만)

> 호출 경로를 가로채는 훅(IAT·인라인 등)을 전제로, 코드/테이블 **무결성·행위**로 탐지하고, 우회(다른 계층·간접호출)·오탐(합법 패치)·성능을 같이 말하며, 클라만 믿지 않고 서버·다계층으로 간다.

D. OS · 저수준 (짧게 나오기 쉬움)

소재	한 줄
유저 vs 커널	유저 훅만으론 부족할 수 있음 → 관측 지점 상향 (우대 경험 연결)
시스템콜 감	경계에서 관측/우회 포인트가 갈림
SEH vs C++ 예외	크래시급은 SEH → `1.11` §9
ASM 한 스니펫	prologue, 간접 `call`, `0xCC` → `1.1`·`1.7`
드라이버 개념	콜백·IOCTL **감만** (구현 문제↓, 우대)

E. 문제 유형 자체가 바뀌는 경우

유형	대비
시나리오 설계 (“이 핵을 어떻게 관측?”)	보안 뼈대 §0
버그 찾기 코드 비중↑	PART D + `1.9`–`1.11`
작은 함수 작성	RAII 핸들, move-only, Rule of 0 (`I01`)
트레이드오프 에세이	탐지율 vs 성능 vs 오탐 vs 호환

F. 수요일 훑기 체크 (깊이 X, 눈만)

- ☐ §0 뼈대 말로 한 번 (후킹 가정 OK)
- ☐ A: 스마트포인터·스레드·무효화·RAII 위치 앎
- ☐ B: DllMain / INVALID / RWX / 호출규약 한 줄
- ☐ C: 후킹·무결성·인젝션·매크로·서버권위 **이름만** 익숙
- ☐ D–E: 유저/커널·SEH·유형 변화 감
- ☐ **다시 PART A C++로 복귀** (본진 잊지 않기)

PART B. C++ 정리 (체크리스트 · Q&A)

> ⛔ 시험 직전 1시간 **제외** (평소·수 D-1 구술용)
>
> 기출에 없는 각도·추가 질문. 기출과 겹치면 **PART A를 먼저**.
> 표기: ★ Pre-Test 필수 · ● 가점/면접

B0. C++11 최소 체크리스트

아래를 **질문만 보고** 말로 설명 → 답 열로 확인. (상세는 B1·PART A)

2.1 수명·소유권

항목	간단 답	☐
RAII	**Resource Acquisition Is Initialization**. 자원 획득=생성자, 해제=소멸자. 스코프 끝나면 자동 정리.	☐
unique / shared / weak	단독소유·복사불가 / refcount 공유 / 카운트 안 올림(순환 끊기).	☐
Rule of 0 / 3 / 5	자원 없으면 0. dtor+copy면 3. move까지면 5. (아래 ★)	☐
복사 vs 이동	복사=내용 복제. 이동=소유권 이전(원본은 빈/미지정 상태).	☐
`=default` / `=delete`	기본 구현 명시 / 호출 금지(복사 막기 등).	☐

★ Rule of 0 / 3 / 5 (한 장)

자원을 **직접** 소유(`new`/`delete`, 핸들 등)하면 특수 멤버를 맞춰라. 가능하면 **0**.

Rule	언제	무엇을
0	`string`/`vector`/`unique_ptr`에 맡김	특수 멤버 **직접 안 씀** → 컴파일러 기본이 올바름
3 (C++98)	raw 자원 소유	**소멸자** + **복사 생성** + **복사 대입** (깊은 복사 또는 `=delete`)

5 (C++11)	3 + 이동	위 3개 + **이동 생성** + **이동 대입**

- 하나라도 직접 정의하면 다른 특수 멤버 생성이 **억제·삭제**될 수 있음 → 세트 단위로 검토.
- 소멸자만/`new`만 있고 복사는 shallow → **double free** (클래스 리뷰 단골).
- 면접 한 줄: "직접 소유하면 3/5, 아니면 스마트포인터·컨테이너로 Rule of 0."

2.2 값 범주·템플릿

항목	간단 답	☐
lvalue / rvalue / xvalue	이름·주소 있는 식 / 임시·이동 대상 / `move` 결과 등(곧 만료).	☐
`move` / `forward`	rvalue로 **캐스트** / 포워딩 참조에서 좌·우 **유지** 전달.	☐
포워딩 참조 vs `U&&`	`T&&`(T 추론)=좌우 둘 다. 구체타입`&&`=우측만.	☐
참조 축소	`&` 하나라도 있으면 `&`. `&&`+`&&`만 `&&`.	☐
`auto` / `decltype`	auto≈초기화로 타입 추론(참조 벗겨질 수 있음). decltype=식의 타입·범주.	☐

2.3 동시성 (기초)

항목	간단 답	☐
data race vs race condition	동기화 없는 충돌 접근→미정의 동작 / 타이밍 논리 버그.	☐
mutex + lock_guard	RAII 락. 소멸 시 unlock.	☐
unique_lock	잠금·해제 유연. condition_variable과 자주.	☐
atomic vs volatile	atomic=원자·동기화. volatile=최적화 억제만(스레드용 아님).	☐
메모리 오더	기본 seq_cst면 안전. volatile로 대체 불가.	☐

2.4 기타 자주 나옴

항목	간단 답	☐
explicit / override / final	암시변환 차단 / 오버라이드 검사 / 더 이상 오버라이드 금지.	☐
nullptr vs NULL/0	포인터 전용 리터럴. NULL·0은 int와 모호.	☐
enum class	스코프+암시 정수변환 제한.	☐

범위 for	`for (auto& x : v)` 이터레이터 루프 단축.	☐
lambda 캡처	`[=]`복사 `[&]`참조. 스레드에 `&`·지역→댕글링.	☐
noexcept와 vector	move가 noexcept여야 재할당 시 이동. 아니면 복사.	☐
미정의 동작 예	댕글링, double free, OOB, data race, 잘못된 delete 짝.	☐

2.5 입으로 연습할 문장 (간단 답)

#	질문	간단 답	☐
1	shared_ptr를 기본으로 안 쓰는 이유?	원자 refcount 비용, 소유권 모호, 순환 참조. 기본은 unique.	☐
2	vector 재할당과 noexcept 이동?	noexcept move면 원소를 이동으로 옮김. 아니면 예외 안전 위해 복사.	☐
3	람다 `&` 캡처 후 스레드?	스레드가 돌 때 원본이 이미 파괴→댕글링. 복사/`shared_ptr` 등.	☐
4	인터페이스에 raw owning pointer?	소유·해제 책임 불명→누수/double free. unique/shared 또는 참조.	☐

B1. 인터뷰 질문 체크리스트 (A~I급)

> 용도: Pre-Test 서술 + **직무면접(Technical Fit)** 구술 공통
> 기준: 경력 → **C++11 Modern 필수** (rvalue/`move`/`forward`), 14/17은 "차이 한 줄"이면 가점.
> 현직자: 신입은 03·언어경험 중심 / TMP·SFINAE 비중 낮음 / 업무 메인=C++ (리버싱·드라이버는 우대). 사용법: **질문만 보고** 60–90초 구술 → 막히면 표의 **답변** 열 확인 → 다시 답하기

자기채점: ☐ 막힘 없이 / △ 답 보고 됨 / ✕ 못함
목 시험 전: **A급 ✕ 0개**, B급은 답변 열을 읽은 뒤라도 입으로 복구

A급 — 반드시 (기출·탈락 포인트)

#	질문	답변	☐
A01	`malloc/free` VS `new/delete`	`malloc`은 메모리만(생성자 X), `new`는 할당+생성자 / `delete`는 소멸자+해제. 타입·실패(`nullptr` VS `bad_alloc`)·배열 짝(`new[]`/`delete[]`)이 다르고 **혼용은 미정의 동작(Undefined Behavior)**.	☐
A02	왜 `new[]`/`delete[]` 짝?	배열은 원소 수·소멸자 반복 호출 정보가 필요. `delete`만 쓰면 소멸자/크기가 깨져 **미정의 동작(Undefined Behavior)**.	☐

A03	`volatile`? 스레드에?	컴파일러 최적화(레지스터 캐싱) 억제. **원자성·메모리 순서 없음** → 스레드 공유는 `std::atomic`/mutex.	☐
A04	`const` vs `constexpr`	`const`=수정 불가(런타임 초기화 가능). `constexpr`=컴파일타임에 알 수 있는 상수/함수. C++11 constexpr 함수는 제약 강함(14에서 완화).	☐
A05	`std::move`는?	**rvalue 참조로 캐스트**할 뿐, 이동을 수행하지 않음. 이동 생성/대입이 불릴 자격만 줌. 이후 원본은 valid but unspecified.	☐
A06	`std::forward`는?	포워딩 참조 `T&&`로 받은 인자의 **값 범주(l/r)를 유지**해 넘김. `forward<T>(x)`.	☐
A07	포워딩 참조 vs rvalue 참조	`template<class T> void f(T&&)`(+추론)=좌/우 둘 다. `void g(U&&)`(U 구체)=우측값만. 이름 있는 `&&` 변수 식은 **lvalue**.	☐
A08	참조 축소	`&`가 하나라도 있으면 `&`. `&&`+`&&`만 `&&`.	☐
A09	lvalue / rvalue	lvalue=이름·주소 잡히는 식. rvalue=임시·이동 대상 등. `std::move(x)`는 xvalue(rvalue 범주).	☐
A10	Rule of 0/3/5	자원 없으면 0(컴파일러 기본). dtor/copy 필요하면 3. move까지면 5. 하나 정의하면 다른 암시 생성이 막힐 수 있음.	☐
A11	복사만 엉성하면?	shallow copy → 소멸 시 **double free**, 대입 누락, 댕글링. Rule of 3/5로 맞추거나 복사 `=delete`+move.	☐
A12	RAII?	**Resource Acquisition Is Initialization**. 자원 획득=생성자, 해제=소멸자. 예: `unique_ptr`, `lock_guard`, Win32 핸들 래퍼. 예외에도 스택 풀며 해제.	☐
A13	unique / shared / weak	unique=단독·복사불가·이동만. shared=refcount(보통 atomic) 공유. weak=카운트 안 올림·순환 끊기. shared를 기본으로 쓰지 말 것(비용·소유 모호).	☐
A14	가상 소멸자 언제?	`Base* p = new Derived; delete p;` 일 때 non-virtual이면 파생 소멸 미호출 → **미정의 동작(Undefined Behavior)/누수**. 다형 삭제 필요 시 기반에 virtual dtor.	☐
A15	클래스 리뷰 렌즈	소유권(shallow/`delete`짝)·Rule5·OOB·불필요 복사·const·explicit·핫패스 할당·가상소멸·싱글톤 dangling.	☐

B급 — 높은 확률 (경력 단골)

#	질문	답변	☐
B01	`=default` / `=delete`	default=컴파일러 기본 구현을 명시. delete=복사/이동 등 **호출 금지**(컴파일 에러).	☐

B02	`explicit`	단일 인자 생성자의 **암시 변환** 차단. 의도치 않은 형변환·오버로드 함정 방지.	☐
B03	`override` / `final`	override=시그니처 불일치 시 컴파일 에러. final=더 이상 오버라이드 불가.	☐
B04	`nullptr` VS `NULL`/`0`	nullptr는 포인터 타입 리터럴. NULL/0은 int와 오버로드 모호.	☐
B05	`enum class`	스코프 있는 열거. 암시적 정수 변환 제한 → 타입 안전.	☐
B06	이니셜라이저 vs 본문 대입	멤버 이니셜라이저=생성과 동시 초기화(효율). 본문 대입=기본생성 후 대입. const/ref는 이니셜라이저 **필수**.	☐
B07	멤버 초기화 순서	**선언 순서** 고정. 이니셜라이저 목록 순서와 무관 → 목록을 선언순에 맞출 것.	☐
B08	값 슬라이싱	파생 객체를 기반 **값**으로 복사하면 파생 부분 잘림. 포인터/참조로 다형 취급.	☐
B09	`noexcept`와 vector	재할당 시 move가 noexcept여야 이동으로 옮김. 아니면 예외 안전 위해 **복사** fallback → 느림.	☐
B10	람다 `[=]`/`[&]`	`[=]`복사 캡처, `[&]`참조. 스레드/콜백에 `&`·지역변수 → **댕글링**.	☐
B11	람다→스레드/콜백	캡처 대상 수명이 콜백보다 길어야 함. `this` 캡처 시 객체 수명. mutable은 const 호출에서도 캡처 수정.	☐
B12	`auto` 주의	기본적으로 참조/top-level const가 벗겨질 수 있음. `vector<bool>` proxy 등. 필요 시 `auto&`/`const auto&`.	☐
B13	`decltype`	식의 타입·값범주를 반영해 선언. `decltype(auto)`는 반환/변수에서 범주 보존에 유용.	☐
B14	오버로드 vs 오버라이드	오버로드=같은 이름·다른 시그니처(컴파일). 오버라이드=가상함수 재정의(런타임, 시그니처 동일).	☐
B15	vtable	virtual 있으면 클래스별 vtable, 객체에 vptr. 호출은 간접. 생성/소멸 중 가상호출은 동적 타입이 기대와 다를 수 있음.	☐
B16	순수 가상 / 추상	`virtual f()=0` → 파생에서 구현 필수, 추상 클래스는 인스턴스화 불가.	☐
B17	다중/가상 상속	다중 상속 가능, diamond는 virtual inheritance. 실무는 복잡도↑ → 인터페이스+구성 선호.	☐
B18	static 멤버	인스턴스와 무관 공유. 보통 TU에서 정의·초기화(C++17 inline static). 생성자에서 “처음 초기화”는 레이스 위험.	☐
B19	연결 / ODR	외부연결=여러 TU 공유 심볼. 내부=`static`/익명네임스페이스. ODR=한 정의 규칙. `inline`으로 헤더 정의 허용.	☐

B20	헤더에 구현	중복정의·빌드시간↑. template/`inline`/header-only는 예외.	☐

C급 — STL · 성능 · 메모리

#	질문	답변	☐
C01	vector / list / deque	vector=연속·랜덤 O(1)·캐시 좋음(기본 선택). list=노드·중간삭제 O(1)이나 캐시↓(practically 거의 안 씀). deque=양끝.	☐
C02	map vs unordered_map	map=정렬 트리 O(log n), < 필요. unordered=해시 평균 O(1), hash+==, 순서 없음.	☐
C03	push_back 시 이터레이터	**재할당** 나면 포인터·참조·이터레이터 전부 무효. `reserve`로 완화.	☐
C04	push_back vs emplace_back	push=임시 만들어 넣기(이동/복사). emplace=인자리스트로 **제자리 생성**(복사 줄일 수 있음).	☐
C05	string 인자	읽기: `const string&` 또는 C++17 `string_view`(수명 주의). 소유 이전: 값+move.	☐
C06	SSO	짧은 문자열을 객체 내부 버퍼에 두어 힙 할당 회피(구현 의존).	☐
C07	복사 줄이기	`const&`/`string_view`, `move`, RVO/NRVO, 불필요 값 전달 금지.	☐
C08	RVO/NRVO	반환 시 임시/로컬을 호출측 자리에 직접 만들어 **복사·이동 생략**. `return std::move(local)`은 오히려 최적화 방해할 수 있음. 상세 → `### 1.5b.`	☐
C09	스택 vs 힙	스택=자동·빠름·크기 제한. 힙=`new`·느림·수동/RAII. AC 핫패스는 힙 할당 자제·사전할당.	☐
C10	alignment	타입별 정렬 요구. 깨면 성능↓·일부 아키 미정의 동작(Undefined Behavior). over-aligned는 맞춤 new.	☐
C11	false sharing	다른 코어가 **같은 캐시라인**의 인접 변수를 쓰면 라인이 핑퐁 → 느림. 패딩/분리.	☐
C12	`inline` 현대 의미	"강제 인라인" 아님. **ODR 허용**(헤더 정의) + 최적화 힌트.	☐

D급 — 동시성

#	질문	답변	☐
D01	data race vs race condition	data race=동기화 없이 충돌 접근 → **미정의 동작(Undefined Behavior)**. race condition=타이밍에 따른 논리 버그(항상 UB	☐

		는 아님).	
D02	mutex + lock_guard	생성 시 lock, 소멸 시 unlock(RAII). 예외에도 해제.	☐
D03	unique_lock / scoped_lock	unique_lock=잠금/해제 유연·CV와 사용. scoped_lock(C++17)=여러 뮤텍스 데드락 없이.	☐
D04	데드락	서로 락을 기다리며 정지. 방지: **전역 락 순서**, 락 최소화, `scoped_lock`, 타임아웃.	☐
D05	atomic	카운터/플래그 등. **volatile 대체 아님**(순서·원자성). 객체 내용 보호는 별도.	☐
D06	메모리 오더	기본 seq_cst면 대체로 안전. acquire/release 최적화는 나중에.	☐
D07	static 초기화·스레드	C++11 함수 내 local static(“magic static”)은 초기화 스레드 안전. 그 전/전역 초기화 순서는 fiasco 주의.	☐
D08	AC 워커 공유 정책	정책값은 mutex 또는 atomic. 모듈 unload 전 워커 **join/stop**, 해제된 상태 접근 금지.	☐

E급 — 미정의 동작(Undefined Behavior) · 버그 헌팅

#	질문	답변	☐
E01	미정의 동작(Undefined Behavior) 예 5개	댕글링, double free, OOB, data race, signed overflow, `new[]`/`delete` 불일치, 초기화 안 된 자동변수 읽기.	☐
E02	소멸 후 참조	dangling → 미정의 동작(Undefined Behavior).	☐
E03	두 번 delete	double free → 미정의 동작(Undefined Behavior).	☐
E04	미초기화 읽기	자동 변수 쓰레기 읽기 → 미정의 동작(Undefined Behavior).	☐
E05	배열 OOB	범위 밖 → 미정의 동작(Undefined Behavior).	☐
E06	const_cast로 const 수정	원본이 const 객체면 수정 → 미정의 동작(Undefined Behavior). const_cast는 비const였던 것만.	☐
E07	placement new	명시적 소멸 후 재사용. 수명 규칙 어기면 미정의 동작(Undefined Behavior).	☐
E08	예외 안전	basic=누수·불법상태 없음 / strong=실패 시 롤백 / nothrow=예외 안 냄. new 중 raw 소유면 누수 → RAII.	☐

F급 — 템플릿 · 형변환

#	질문	답변	☐
F01	함수 vs 클래스 템플릿	둘 다 타입 매개변수. 함수는 인자로 추론 잘 됨. 정의는 보통 헤더.	☐
F02	특수화	클래스=전체/부분 특수화. 함수는 특수화보다 오버로드가 흔함.	☐
F03	캐스팅 4종	static=관련 타입·컴파일타임. dynamic=다형 계층 안전 다운(RTTI, 실패 nullptr). const=const성 제거. reinterpret=비트 재해석·최후수단.	☐
F04	CRTP	기반이 파생을 템플릿 인자로 → 정적 다형성. 가상 없이 확장, 핫패스 이득 가능.	☐
F05	SFINAE	*(우선순위↓ 현직자: TMP 크게 중요 X)* 치환 실패≠컴파일 에러. `enable_if`. 나오면 한 줄만. Pre-Test 전 스킵 OK.	☐

G급 — C++14/17 한 줄

#	질문	답변	☐
G01	C++14 constexpr	함수 제약 완화(로컬 변수·루프 등).	☐
G02	`make_unique`	unique_ptr 예외안전 생성(C++14).	☐
G03	`string_view`	비소유 뷰. **원본 수명** 필수.	☐
G04	optional/variant/any	값 없음 / 대안 타입 하나 / 타입 소거.	☐
G05	구조화 바인딩	`auto [a,b] = pair;`	☐
G06	`if constexpr`	컴파일타임 분기(템플릿에서 특히).	☐
G07	본인 표준	예: “실무 C++11/14 중심, 이동·스마트포인터·RAII. 17 문법은 팀 툴체인 따라. 11은 확실히.”	☐

H급 — Anti-Cheat / 서류 연계

#	질문	답변 (구술 예시)	☐
H01	모듈 버퍼 소유권	CDN 복호화 버퍼·모듈은 `unique_ptr` 등 **단일 소유**. 하위로 `move` 또는 비소유 참조/`get()`. 복사·이중 free 방지.	☐
H02	핫패스 raw new 기피	할당·예외·단편화·누수. 사전 할당/풀, 스택·멤버 버퍼.	☐
H03	체크썸 워커 공유	정책·기준값은 mutex 또는 atomic. on/off·임계값 동기화. unload 시 stop+join.	☐

H04	DLL unload 수명	정적·스레드·콜백이 언로드된 코드/데이터를 건드리면 AV. 등록 해제·join 후 FreeLibrary.	☐
H05	게임 안 예외	SDK 경계에서 catch·에러코드. 소멸자·이동은 noexcept. 프로세스 abort 정책은 제품 합의.	☐
H06	Pimpl	구현 숨김 → ABI 안정·헤더 의존↓·재컴파일 감소. 보안 SDK 공개 헤더에 유리.	☐
H07	Win32 핸들 RAII	custom deleter `unique_ptr` 또는 전용 클래스. 복사 금지·이동, 소멸에서 `CloseHandle`.	☐
H08	훅과 C++ 객체	멤버함수는 this+호출규약. 일반 함수 포인터와 다름. 트램폴린/thunk 고려.	☐

I급 — 화이트보드 / 라이브코딩

시간 있으면 **직접 손코딩**. 목 전까지 2개만 끝까지.

#	과제	답(합격 기준·요지)	☐
I01	HANDLE RAII	멤버 `HANDLE`, dtor `CloseHandle`. copy `=delete`, move 시 원본 `INVALID`.	☐
I02	unique_ptr 넘기기	`void f(unique_ptr<T>)` 또는 `unique_ptr<T>&&` + `std::move`. raw `get`만 넘기면 소유권 애매.	☐
I03	thread-safe 카운터	`atomic<int>` 증감 또는 mutex 보호. volatile 금지.	☐
I04	volatile 플래그 수정	`atomic<bool>` (+ 필요 시 memory order).	☐
I05	Rule of 5 buffer	raw `char*`면 dtor/`delete[]`, copy deep, move는 포인터 이전 후 `nullptr`. 또는 `vector`로 Rule of 0.	☐
I06	push vs emplace	큰 객체: emplace가 임시+move를 줄일 수 있음. 이미 만든 임시면 `push_back(std::move(x))`.	☐
I07	가상소멸 버그	`Base*`(non-virtual dtor)로 `Derived` delete → 지적 + virtual 추가.	☐
I08	읽기 전용 문자열 API	`void f(string_view)` 또는 `const string&`. view는 인자 수명이 호출 동안 유효해야.	☐

인터뷰 답변 템플릿 (C++ 문항 공통)

1) 한 줄 정의
2) 비슷한 것과 차이 (표로 말하기)

3) 안 지키면 생기는 일 (미정의 동작(Undefined Behavior)/버그/성능)
4) 실무·AC에서 언제 쓰는지 (한 문장)

약할 때 솔직 버전

> "실무에서는 C++11 기준으로 소유권·이동·동시성 기초를 씁니다. OOXX는 깊게 안 써봤고, 지금은 이렇게 이해하고 있습니다: …"

B2. 기본 Q&A (포인터 · OOP · STL · 스레드)

외부 C++ 인터뷰 목록을 KSS Pre-Test용으로 정리.
★ = 목 시험·기출 직결 · ● = 나오면 가점 · (겹) = §1/§2A와 중복 → 여기 답만 복습

2C.1 C++ 기본

★ 레퍼런스와 포인터의 차이점?

	포인터 T*	참조 T&
널	가능 (nullptr)	보통 불가(바인딩 필수)
재바인딩	다른 주소로 변경 가능	초기화 후 재바인딩 불가
문법	* / ->	별칭처럼 사용
산술	포인터 산술 가능	없음
다중 간접	T**	참조의 참조는 축소 규칙

한 줄: 참조는 "반드시 유효한 객체에 붙는 별칭", 포인터는 "주소 값(널·재지정 가능)".

★ 스택과 힙에서의 메모리 할당 차이점?

	스택	힙
할당	자동(로컬 변수), 빠름	new/malloc, 상대적으로 느림
해제	스코프 종료 시 자동	수동 또는 스마트포인터
크기	제한적(스레드당)	큼
수명	스코프	프로그래머가 결정
실패	스택 오버플로	bad_alloc / nullptr

AC: 핫패스·프레임 중 힙 할당 최소화, 큰 버퍼는 사전 할당/풀.

★ 어떤 종류의 스마트 포인터가 있어?

- `unique_ptr` — 단독 소유
- `shared_ptr` — 공유 소유 + refcount
- `weak_ptr` — `shared_ptr` 관찰, 순환 참조 끊기
- (C++98) `auto_ptr` — **deprecated/삭제**, 언급만·쓰지 말 것

★ `unique_ptr`은 어떻게 구현돼? 하나만 소유하게 하는 법은?

- 내부적으로 대략 **raw pointer + custom deleter**
- **복사 생성/대입** `= delete` → 복사 불가
- **이동 생성/대입**만 허용 → 소유권 이전 시 원본은 `nullptr`
- 소멸자에서 `deleter(ptr)`
  → "하나의 객체만 소유" = **복사 금지 + 이동 시 독점 유지**

★ `shared_ptr`은 어떻게 작동해? refcount 동기화는?

- 제어 블록: **strong count** (+ weak count), 관리 객체 포인터
- 복사 시 strong++, 소멸/reset 시 strong-- → 0이면 객체 delete
- weak==0이면 제어 블록 해제
- **동기화:** 카운터는 보통 **atomic** 증감 (구현 정의, MSVC/libstdc++ 등 atomic).
  가리킨 객체 T 자체의 스레드 안전은 보장 안 함 → T 접근은 별도 동기화
- `make_shared`면 제어블록+객체를 한 번 할당(효율)

★ `unique_ptr`을 복사하거나 다른 객체로 넘길 수 있어?

- **복사: 불가** (컴파일 에러)
- **이동: 가능** — `std::move`, 또는 함수가 `unique_ptr`을 값으로 받아 이동
- raw로 넘기면 소유권 애매 → `get()`은 비소유 관찰용, `release()`는 소유권 포기

★ rvalue와 lvalue는 뭐야? /(겹) §1.5–1.6

- **lvalue:** 이름 있는 객체, 주소 잡히는 쪽 (대입 왼쪽 등)
- **rvalue:** 임시·이동 가능한 값 (리터럴, 임시 객체 등). xvalue = "곧 만료되는" 이동 대상
- `T&&`에 바인딩, `std::move`가 lvalue를 rvalue처럼 취급하게 캐스팅

★ `std::move`와 `std::forward`는 뭐야? /(겹) §1.5

- `move`: **rvalue 참조로 캐스트** (이동 강제 아님)
- `forward`: 포워딩 참조에서 **값 범주 유지** 전달

2C.2 OOP

● 클래스의 private 필드에 접근하는 방법?

- 같은 클래스 멤버 함수 / `friend` 함수·클래스
- public 게터(필요 시)
- 포인터 강제 캐스팅 등은 **미정의 동작(Undefined Behavior)·하지 말 것** (면접에서 "하면 안 된다"고 말기)

● 클래스가 여러 클래스를 상속받을 수 있어?

- **가능** (다중 상속). diamond는 **virtual inheritance**
- 실무·AC SDK에선 복잡성↑ → 인터페이스(순수가상) + 구성 선호이 흔함

● static 필드는 클래스 생성자에서 초기화돼?

- **아니요.** 인스턴스 생성자에서 "처음 초기화"하는 패턴은 위험(레이스).
- **정의(translation unit)에서 초기화**, `inline static`(C++17), 또는 함수 내 local static
- const integral static은 클래스 안 초기 초기화 가능(구규칙). 그 외는 보통 바깥 정의

★ 생성자/소멸자에서 예외가 발생할 수 있어? 어떻게 막아?

- **생성자:** 예외 가능 → 객체는 "생성 실패", 이미 생성된 **멤버·베이스는 역순 소멸**. raw `new` 멤버는 누수 위험 → **RAII/스마트포인터**
- **소멸자:** 예외가 소멸자 밖으로 나가면(특히 이미 예외 전파 중) → **`std::terminate`.**
  막기: 소멸자는 `noexcept`(기본), 내부에서 예외 삼키거나 로그만, 실패는 에러 코드/플래그로

★ 가상 메서드는 뭐야?

- 파생에서 **오버라이드** 가능한 멤버. 기반 포인터/참조로 호출해도 **동적 바인딩**(vtable)
- `virtual`, 파생에 `override` 권장

★ 가상 소멸자가 왜 필요해? /(겹) A14

- `Base* p = new Derived; delete p;` 일 때 소멸자가 non-virtual이면 **파생 소멸자 미호출 → 미정의 동작(Undefined Behavior)/누수**
- 다형 삭제가 필요하면 기반에 `virtual ~Base()`

● 추상 클래스와 인터페이스의 차이점?

- C++에 `interface` 키워드 없음
- **추상 클래스:** 순수 가상(`=0`) ≥1 → 인스턴스화 불가. 상태·구현 일부 가능
- **인터페이스처럼 쓸 때:** 순수 가상만, 데이터 멤버 최소/없음, (보통) 가상 소멸자
  → "인터페이스 = 순수 추상 클래스"로 말하면 됨

● 생성자를 가상으로 만들 수 있어?

- **불가.** 생성 중에는 아직 동적 타입이 완성되기 전.
- 대안: 팩토리, `clone()` 가상 메서드, CRTP

★ const 키워드는 클래스 메서드에서 어떻게 사용돼? /(상세 §1.4b)

- `void foo() const` — **비트 수준으로 멤버를 수정하지 않음**(논리적 const). const 객체에서 호출 가능
- `mutable` 멤버는 const 메서드에서도 변경 가능(캐시·mutex 등)
- const 오버로드: `T& operator[]` VS `const T& operator[] const`
- 포인터·move·생성자·`const_cast`까지 → `### 1.4b` **const 특집**

★ 객체를 복사로부터 보호하는 방법?

1. 복사 생성/대입 `= delete` (C++11)
2. (구식) private이고 정의 안 함
3. 이동만 허용 (`unique_ptr` 패턴)
4. 싱글톤·리소스 핸들 클래스에 흔함

2C.3 STL 컨테이너

★ vector와 list의 차이점? /(겹) C01

	`vector`	`list` **(연결리스트)**
메모리	연속	노드 분산
랜덤액세스	O(1)	O(n)
중간 삽입/삭제	O(n) (이동)	O(1) (위치 알 때)
캐시	우수	나쁨

실무 기본은 **`vector`**. `list`는 거의 안 씀.

★ map과 unordered_map의 차이점? /(겹) C02

	`map`	`unordered_map`
구조	트리(보통 RB)	해시 테이블
복잡도	O(log n)	평균 O(1), 최악 O(n)
순서	키 정렬	없음
요구	`operator<` 또는 comparator	`std::hash<Key>` + `==`

★ `push_back()` 때 vector 이터레이터가 무효화되는 경우? /(겹) C03

- **재할당이 일어나면** 모든 포인터·참조·이터레이터 무효
- 용량 부족 시 `capacity` 증가 → 재할당
- `reserve`로 미리 확보하면 `push_back`만으로 무효화 안 할 수 있음
- 재할당 없어도 **끝 삽입**은 end()만 영향 주는 구현이지만, **안전하게 "재할당 가능 = 전부 무효"**로 답하는 편이 좋음

● map/unordered_map에서 쓰도록 클래스를 수정하는 방법?

`map<Key,V>` 키:

- `bool operator<(const Key&) const` 또는 외부 comparator
- 동등성: `!(a<b) && !(b<a)`

`unordered_map<Key,V>` 키:

- `bool operator==(const Key&) const`
- `std::hash<Key>` 특수화 또는 커스텀 hash를 템플릿 인자로
- **같으면 해시 같아야** (hash/equals 일관성)

값(mapped type)은 복사/이동 가능하면 보통 OK.

2C.4 스레드

★ 프로세스와 스레드의 차이점? /(겹) §2B.1

- **프로세스:** 독립 주소공간, 커널이 스케줄하는 실행 단위의 컨테이너
- **스레드:** 같은 프로세스 **주소공간 공유**, 스택·레지스터는 스레드별
- 통신: 스레드=공유메모리(+동기화), 프로세스=IPC

● 같은 스레드를 두 번 실행할 수 있어?

- `std::thread` 객체: **한 번 join/detach 후 다시 join 불가**.
- 같은 함수를 **새 `std::thread`로 다시 시작**하는 건 가능 (새 실행 단위)
- 종료된 네이티브 스레드를 "같은 핸들로 재실행"은 보통 불가 → 새로 생성
- `std::thread::id` / `joinable()` 로 상태 확인

★ 스레드를 동기화하는 방법? /(겹) D급

- `mutex` + `lock_guard`/`unique_lock`
- `std::atomic`
- condition_variable, SRWLock/CriticalSection(Win)

- (고급) lock-free, 메시지 큐로 공유 최소화

★ 데드락은 뭐야? /(겹) D04

- 둘 이상 스레드가 **서로의 락을 기다리며** 영구 대기
- 필요 조건 감: 상호배제, 점유대기, 비선점, 순환대기
- 방지: **락 전역 순서**, 한 번에 하나, `scoped_lock` 다중락, 타임아웃, 락 제거 설계

B3. 빌드 · 링킹 · 캐스팅 · 복사 · 다형성

외부 면접 노트에서 **KSS Pre-Test / AC**에 쓸 것만 압축.
스킵: Python 비교, SOLID 장문, 블로그 링크 모음, 이미 §1·§2C와 100% 겹치는 장황 설명
★ 목 시험 추천 · ● 나오면 가점

2D.1 ★ 빌드 과정 (전처리 → 컴파일 → 어셈블 → 링크)

시험 한 줄 순서: **소스 → 전처리 → 컴파일(→어셈블) → 오브젝트(.o/.obj) → 링킹 → 실행파일/DLL**

단계	하는 일	산출물
전처리	`#include` 삽입, `#define` 매크로 치환, `#if` 조건부 컴파일	전처리된 번역 단위 (거대한 .ii 등)
컴파일	구문·의미 분석, IR, 최적화, (보통) 어셈블 생성까지	어셈블리 또는 바로 오브젝트
어셈블	어셈블리 → 기계어 오브젝트. 요즘 툴체인은 컴파일에 묶이는 경우가 많음	`.o` / `.obj`
링킹	심볼 해석, 여러 오브젝트+라이브러리 결합, 재배치	`.exe` / `.dll` / `.so`

링킹에서 일어나는 일: 미정의 참조 해결, 정적 라이브러리 끌어오기, 전역/정적 초기화 심볼 배치 등. **컴파일러마다** GCC/Clang/MSVC 플래그·확장자는 다르다고 한 줄 덧붙이면 가점.

AC 연결: 헤더 오염·매크로·ODR, 보호/난독화는 **링크 이후 바이너리**에 적용하는 경우가 많음(본인 CI 보호 단계).

★ static linking vs dynamic linking

	Static	Dynamic (DLL/so)
시점	링크 시 라이브러리 코드 포함	로드/런타임에 해석·바인딩

실행파일 크기	커짐, 의존 DLL 적을 수 있음	작아질 수 있음, DLL 필요
업데이트	라이브러리 바꾸면 **재링크**	DLL 교체로 갱신 가능(호환 주의)
성능	호출이 대체로 단순	IAT/지연 로드 등 한 단계
실패 모드	링크 시 심볼 부족	로드 실패, 버전 불일치

AC: 보안 SDK를 DLL로 배포·CDN 업데이트 = **dynamic** 쪽. 후킹 관측도 IAT/지연 로드와 연결.

2D.2 ★ `endl` vs `\n`

	`\n`	`std::endl`
줄바꿈	O	O
버퍼 flush	X (기본)	**O**
성능	대량 출력에 유리	잦으면 느림

로그·즉시 확인이 필요하면 `endl`/명시 `flush`, 핫패스·대량이면 `\n`.
AC 클라 로그 스팸에 `endl` 남발 = 프레임 스파이크 원인 될 수 있음.

2D.3 ● ASCII / 현대 인코딩 (짧게)

- ASCII: 7비트, 128문자, 영어·제어문자 중심
- 현대: **UTF-8** 등 유니코드. UTF-8은 ASCII 범위와 **바이트 호환**
- Windows API: `A` vs `W`(UTF-16). 경로·CreateFile은 **Unicode(`W`)** 가 안전 (§ 1.9와 연결)

2D.4 OOP · 다형성 (압축)

● OOP 한 줄 + 4대 특성

상태+행위 객체와 상호작용. **추상화 / 캡슐화 / 상속 / 다형성**.
(SOLID는 직무면접용 ○ — Pre-Test에 장문 불필요. 이름만: SRP OCP LSP ISP DIP)

★ 컴파일타임 다형성 vs 런타임 다형성

	컴파일 타임	**런타임**
결정 시점	컴파일	실행
C++	**오버로딩**, **템플릿**	**virtual** + 기반 포인터/참조
장점	인라인·최적화 여지	확장·플러그인형 유연

런타임에 virtual+포인터/참조+override가 합쳐져야 “동적 바인딩”이 의미 있음.

★ 가상함수 / 왜 전부 virtual 안 쓰나

- vtable + vptr, 호출이 간접 → **오버헤드**(작지만 핫패스에선 의미)
- C++는 기본 non-virtual (Java와 다름)
- AC/게임: 핫패스 가상 호출 자제, 필요 시에만

★ 가상 소멸자 /(겹) — 상속+`Base*` delete 시 누수·미정의 동작(Undefined Behavior) 방지

● 순수 가상 vs 일반 가상

- `= 0` → 반드시 파생에서 구현, 클래스 추상(인스턴스 불가)
- 일반 virtual → 오버라이드 선택

● class vs struct

기본 접근만 다름: **class=private**, **struct=public**. 기능적으로 동일에 가깝다.

● 오버로딩 vs 오버라이딩

- 오버로딩: 같은 이름, **시그니처 다름**, 같은 스코프(컴파일)
- 오버라이딩: 가상함수 **재정의**, 시그니처 동일(런타임)

● 암시적 특수 멤버 함수

디폴트/복사 생성·복사대입·(C++11) 이동 생성·이동대입·소멸.
하나 직접 정의하면 다른 것 생성이 **억제·삭제**될 수 있음 → Rule of 0/3/5.

2D.5 ★ 메모리 · 스마트포인터 보강

★ 더블프리 / 댕글링 / 누수

용어	상황
Double free	같은 주소 `delete`/`free` 두 번
Dangling	해제·만료된 주소를 포인터가 가리킴
Leak	할당 후 해제 경로 없음

예방: **RAII**, `unique_ptr`/`shared_ptr`, 소유권 하나.

★ RAII (Resource Acquisition Is Initialization)

획득=생성자, 해제=소멸자. 예: 스마트포인터, `lock_guard`, Win32 핸들 래퍼.

● `enable_shared_from_this`

`this`로 새 `shared_ptr`를 만들면 **별도 제어블록** → double free.
이미 `shared_ptr`로 관리 중일 때 `shared_from_this()` 사용.

★ 얕은 복사 vs 깊은 복사

- 얕은: 포인터 주소만 복사 → double free/dangling
- 깊은: 가리킨 자원까지 새로 복사
- 대안: 복사 금지 + move, 또는 공유는 `shared_ptr`

● malloc / calloc / realloc 보충 /(겹) §1.2

	역할
`malloc`	미초기화 블록
`calloc`	0 초기화
`realloc`	크기 변경(이동 가능). **실패 시 NULL인데 옛 포인터 덮어쓰면 누수** → 임시 포인터에 받기

객체(생성자 필요)는 `new`/`delete`. POD·C API 연동만 malloc 계열.

● 멤버 이니셜라이저 vs 본문 대입

생성 시 초기화가 효율. **const·참조 멤버는 이니셜라이저 필수**.

2D.6 ★ 캐스팅 · 업/다운캐스트

업캐스트 / 다운캐스트

- **업:** Derived→Base, 안전·암시적 가능
- **다운:** Base→Derived, 위험 → 실제 타입 맞을 때만. `dynamic_cast`(+virtual 필요) 실패 시 `nullptr`

★ 캐스팅 4종

캐스트	용도
`static_cast`	관련 타입, 컴파일타임. 잘못된 다운은 미정의 동작(Undefined Behavior) 가능
`dynamic_cast`	다형 계층 안전 다운. RTTI, 상대적 비용

`const_cast`	const/volatile 제거·추가. **원본이 const면 수정 미정의 동작(Undefined Behavior)**
`reinterpret_cast`	비트 재해석. 최후 수단

AC/성능 핫패스: 타입을 알면 `static_cast`, 불확실하면 `dynamic_cast`.

2D.7 이동 · 참조 · 기타

★ 이동 의미론 /(겹) §1.5

깊은 복사 대신 **리소스 소유권 이전**. 이동 생성/대입 + `std::move`. 원본은 valid but unspecified.

● 참조 반환

복사 없이 원본 접근. **지역 변수 참조 반환 = 댕글링**. 멤버·인자로 받은 참조 등 수명이 호출자보다 길 때만.

● 상등(Identity) vs 동등(Equality)

- 상등: 같은 주소(`&a == &b`)
- 동등: 값 같음(`operator==`)
  `unordered_map` 키는 hash와 `==`(동등) 일관성.

● 포인터 vs 배열 (한 줄)

배열 이름은 감쇠해 포인터처럼 쓰이지만 **고정 크기·재할당 불가**. 포인터는 재지정·산술 가능.

● 템플릿 (한 줄)

타입 매개변수로 컴파일타임 코드 생성 = 컴파일타임 다형성. 정의는 보통 헤더.

● 참조 리턴·해시맵 /(겹)

`unordered_map`≈해시, `map`≈균형트리 O(log n) — §2C.3과 동일.

B4. C++03 위주 인터뷰 (프리 C++11 기초)

> 현직자: **신입 ≈ C++03, 경력 ≈ C++11 Modern**.
> 경력도 리뷰·구술은 여전히 **03 뼈대**(가상/복사/RAII/new) 위에 11(move/forward)을 얹는 형태가 많음.
> 여기 = “11 이전에도 있던 것” + “03에서 어떻게 말했는지”. 11 전용은 PART A·B1.

목 Pre-Test (경력): B4는 ★만 훑기 → 시간은 **1.5–1.6b move/forward**에. 깊게는 직무면접용.

B4.0 C++03 vs C++11 — 없는 것 / 대신 쓰던 것

C++11+	C++03에서
`std::move` / rvalue ref / Rule of 5	**없음**. 복사가 기본. Rule of **3**
`nullptr`	`0` / `NULL` (오버로드 모호 주의)
`override` / `final`	없음. 시그니처 실수는 런타임에 다른 함수로 숨을 수 있음
`=default` / `=delete`	복사 금지는 **private + 선언만** (정의 안 함)
`unique_ptr` / `shared_ptr`	`auto_ptr`(위험·비권장), 직접 RAII, Boost
`auto` (타입추론)	없음 (저장소 기간 `auto`는 다른 의미·구식)
lambda	함수 객체(functor) / 함수 포인터
`constexpr`	`const` + 열거/매크로/템플릿로 상수 표현
range-for	이터레이터 루프
`unordered_map`	`std::map` 또는 TR1/`hash_map`(비표준)
스레드 표준	플랫폼 API / Boost.Thread

면접에서: "C++03만 알면?" → **부족. 11 이동·스마트포인터는 필수.**
"03을 몰라도?" → **가상/복사/RAII/STL 기초 모르면 탈락.**

B4.1 ★ Rule of Three (C++03 핵심)

자원을 소유하는 클래스면 보통 셋이 필요:

1. **소멸자** — 해제
2. **복사 생성자** — 깊은 복사
3. **복사 대입 연산자** — 자기대입 주의, 깊은 복사

하나라도 직접 쓰면 나머지도 검토. (C++11에선 여기에 **이동** 둘 → Rule of 5)

복사 금지 (03 관용구)

```
class NonCopyable {
private:
    NonCopyable(const NonCopyable&);            // 선언만
    NonCopyable& operator=(const NonCopyable&);
};
```

C++11: `= delete`.

자기대입

```
Foo& operator=(const Foo& o) {
    if (this != &o) { /* 복사 */ }
    return *this;
}
```

B4.2 ★ 인터뷰 Q&A 표 (C++03 단골)

#	질문	답변	☐
03-01	포인터 vs 참조	포인터=널·재지정·산술 가능. 참조=별칭, 초기화 필수, 재바인딩 불가(사실상).	☐
03-02	`new`/`delete` vs `malloc`/`free`	new는 생성자, delete는 소멸자. 혼용 미정의 동작. 배열은 `new[]`/`delete[]`.	☐
03-03	가상 함수 / 동적 바인딩	기반 포인터·참조로 파생 구현 호출. vtable.	☐
03-04	가상 소멸자	`Base*`로 `Derived` delete 시 필요. 없으면 파생 소멸 생략·누수/미정의 동작.	☐
03-05	순수 가상 / 추상 클래스	`virtual f()=0`. 인스턴스화 불가. 인터페이스 역할.	☐
03-06	오버로드 vs 오버라이드	오버로드=시그니처 다름(컴파일). 오버라이드=가상 재정의(런타임).	☐
03-07	업캐스트 / 다운캐스트	업=안전·암시적. 다운=`dynamic_cast`(다형 필요) 또는 위험 감수 `static_cast`.	☐
03-08	값 슬라이싱	파생을 기반 **값**으로 복사하면 잘림. 포인터/참조로 다형.	☐
03-09	얕은 복사 vs 깊은 복사	주소만 vs 내용 새 할당. 얕으면 double free.	☐
03-10	RAII	생성자에서 자원 획득, 소멸자에서 해제. 예외에도 스택 풀며 정리.	☐
03-11	`explicit`	단일 인자 생성자 암시 변환 방지.	☐
03-12	`mutable`	`const` 멤버 함수 안에서도 수정 가능한 멤버(캐시·mutex 카운트 등).	☐
03-13	`static` 멤버	인스턴스 공유. cpp에 정의.	☐

03-14	멤버 초기화 순서	**선언 순서**. 이니셜라이저 목록 순서 아님.	☐
03-15	const / ref 멤버	생성자 **멤버 이니셜라이저**로만 초기화.	☐
03-16	`vector` VS `list` VS `deque`	연속·랜덤 / 노드·중간삭제 / 양끝. 실무 기본은 vector.	☐
03-17	`map`	키 정렬 트리(보통 RB), O(log n), `operator<`.	☐
03-18	이터레이터 무효화	vector 재할당 시 포인터·이터레이터 무효.	☐
03-19	`volatile`	그 변수 접근 최적화 억제. **스레드 안전 아님**.	☐
03-20	예외 / 소멸자	소멸자에서 예외가 밖으로 나가면 위험(terminate). 소멸자는 삼키거나 안 던짐.	☐
03-21	캐스팅 4종	static / dynamic / const / reinterpret — B3와 동일(98부터 존재).	☐
03-22	템플릿	타입 매개변수, 헤더에 정의, 컴파일타임 다형성.	☐
03-23	`auto_ptr`	복사 시 소유권 이동처럼 동작 → 컨테이너에 넣으면 위험. **지금은 unique_ptr**. “03 유산·쓰지 말 것”.	☐
03-24	NULL vs 0	정수와 오버로드 모호. C++11은 `nullptr`.	☐
03-25	class vs struct	기본 접근만 private vs public.	☐
03-26	friend	private 접근 허용. 캡슐화 깨지니 최소 사용.	☐
03-27	다중 상속 / diamond	가능. diamond는 virtual inheritance. 복잡도↑.	☐
03-28	함수 객체 (functor)	`operator()` 오버로드. 람다 이전의 “상태 있는 콜백”.	☐
03-29	임시 객체 수명	참조에 바인딩되면 그 스코프까지 연장(상수 참조 등). 댕글링 주의.	☐
03-30	POD / 트라이비얼 (감)	C처럼 복사 가능한 단순 타입. malloc 등과 연관해 언급.	☐

B4.3 ★ C++03 스타일 클래스 리뷰 체크 (기출 10번과 연결)

코드 리뷰할 때 03 기준으로 먼저 볼 것:

1. 소멸자만 있고 복사 생성/대입 없음 → shallow + double free
2. `new`/`delete` 짝, `new[]`/`delete[]`
3. 가상 소멸자 없는 다형 계층
4. 대입 연산자 자기대입 미처리
5. 예외 시 raw 포인터 누수 (RAII 없음)
6. 값으로 파생 전달 → 슬라이싱
7. `auto_ptr` / 소유권 불명 포인터 멤버

그다음 C++11 렌즈: move, `=delete`, `unique_ptr`, `override`.

B4.4 구술 연습 문장 (03 → 11로 업그레이드)

1. "C++03에선 Rule of 3로 복사·소멸을 맞췄고, 지금은 이동까지 Rule of 5와 `unique_ptr`로 소유권을 단일화합니다."
2. "널은 예전에 `NULL`/`0`이었고 오버로드 문제가 있어 지금은 `nullptr`을 씁니다."
3. "콜백은 functor나 함수 포인터였고, 단순하면 람다, 수명·소유가 있으면 `function`/명시적 객체로 갑니다."
4. "복사가 비싸면 03에선 참조로 넘기거나 복사를 막았고, 지금은 이동으로 소유권을 넘깁니다."

B4.5 목 시험에서의 위치

우선	내용
★★★	Rule of 3, 가상소멸, new/delete, 포인터/참조, 얕은/깊은 복사
★★	RAII, vector 무효화, map, explicit, 이니셜라이저
★	auto_ptr 경고, NULL, functor (알아만)
직무면접	diamond, friend, 임시 수명, POD

C++11 문항(move/forward/volatile≠atomic)과 **겹치면 11 답에 03 배경을 한 줄 보태면** 가점.

PART C. Windows · OS · 공고 요건

> 기출 CreateFile/호출규약(PART A)과 별도로, **공고 필수/우대** 정리.
> Pre-Test용 OS는 부록 Sir.LOIN의 ★ + PART A 1.7~1.9.
> 현직자: 업무 메인=**C++ 유저 라이브러리(+커널)** 핵 탐지. 리버싱·asm·드라이버는 **우대≠필수**.

C1. 공고 필수요건별 준비

3.1 C++ 기반 소프트웨어 개발 역량

- 위 §1, §2 중심 — **경력 최소 = C++11** (rvalue/`move`/`forward`, RAII, 스마트포인터)
- 추가: STL 복잡도 (`vector`/`map`/`unordered_map`), 반복자 무효화
- AC 맥락: 핫패스에서 할당·락·예외 최소화
- TMP/SFINAE 심화는 필수로 보지 않음 (현직자)

3.2 Windows 클라이언트 개발 / 내부 동작

준비 항목:

주제	꼭 말할 수 있어야 하는 것
프로세스/스레드	PEB/TEB 감, 생성 API, TLS
가상 메모리	VA, page, commit/reserve, `VirtualAlloc`, 보호 속성 RWX
동기화	CriticalSection, SRWLock, Event, 커널 객체 vs 유저
IPC	파이프 vs 공유메모리 → `### 3.2b`
DLL	로드 순서, DllMain 제약, loader lock
PE	DOS/NT 헤더, Section, IAT/EAT, TLS 콜백, ASLR/DEP 관계

예상 질문

- DLL을 `LoadLibrary`로 올리면 어떤 일이 순서대로 일어나는가?
- DllMain에서 하면 안 되는 일은?
- 실행 파일 로딩 과정(매핑 → import 해석 → TLS → entry)을 단계로
- 프로세스 간 통신에 파이프와 공유메모리 중 무엇을 고를까? (차이)

3.2b ★ IPC: 파이프 vs 공유메모리

⏱️ 직전1h · 48–54분(표)

IPC = Inter-Process Communication. 프로세스는 주소 공간이 달라 **포인터를 그대로 공유할 수 없음** → 커널이 중개하는 수단이 필요.

	파이프 (Pipe)	공유 메모리 (Shared Memory)
한 줄	바이트 **스트림**(큐에 넣고 빼기)	같은 **물리 페이지**를 각자 VA에 매핑
모델	쓰기 프로세스 → (커널 버퍼) → 읽기 프로세스	양측이 같은 버퍼를 **직접** 읽/쓰

Windows 감	Anonymous pipe / **Named Pipe** (`CreateNamedPipe` / `CreateFile`)	파일 매핑 (`CreateFileMapping` + `MapViewOfFile`), 섹션 객체
복사	보통 **커널 경유 복사**(유저→커널→유저)	매핑 후엔 **복사 없이** 접근 (초기 세팅·동기화 비용은 별도)
동기화	읽기/쓰기가 자연스럽게 **흐름·블로킹**으로 맞춤	**필수 별도** (Mutex, Event, 인터락…). 없으면 data race
메시지 경계	바이트 스트림이면 경계는 앱이 정함. Named pipe는 메시지 모드 가능	레이아웃·프로토콜을 앱이 정함 (패딩·버전 주의 → `1.11`)
속도 감	소량·요청/응답·단순 채널에 적합	**대량·저지연** 공유에 유리 (한 번 매핑 후)
수명·연결	핸들·연결 지향에 가깝게 쓰기 쉬움	매핑 해제·객체 수명·이름 있는 매핑 권한 주의
보안 AC 감	파이프 이름·접근권한·스푸핑	매핑 이름·ACL, **다른 프로세스가 RWX로 붙는지**, 치트가 공유영역 조작

비유

- 파이프 = **택배(복사해서 전달)**. 순서·흐름은 쉽고, 대량이면 비용↑.
- 공유메모리 = **같은 화이트보드**. 빠르지만 **누가 언제 쓰는지 규칙(락)** 없으면 깨짐.

고르는 감 (면접 한 줄)

- 명령·로그·RPC풍 **메시지** → 파이프(또는 소켓)가 단순.
- 프레임·대용량 버퍼·상태 테이블 **공유** → 공유메모리 + 동기화.
- AC: 타 프로세스와 공유하는 순간 **신뢰 경계** — 입력 검증·권한·레이아웃 변조를 같이 말함.

스레드 vs 프로세스 (한 줄)
같은 프로세스 스레드 = 이미 공유 주소공간(+동기화).
다른 프로세스 = 위 IPC. “스레드 공유메모리”와 “프로세스 공유메모리”를 용어로 섞지 말 것.

3.3 x64 Assembly / 디스어셈블리 분석

- 호출 규약(§1.7) + prologue/epilogue
- `call`/`ret`, RIP-relative addressing
- 간접 호출 vs 직접 호출 (훅 포인트)
- 조건부 점프와 패치
- IDA/x64dbg에서 “이 함수가 인자를 어떻게 쓰는가” 읽기 연습

연습: 본인 포트폴리오 메신저/매크로 관련 — `GetDisplayName` / `PrintWindow` 근처 디스어셈을 다시 읽어보기

3.4 OS 핵심: 커널·드라이버·시스템콜·인터럽트·메모리

- 유저↔커널 전환, syscall gate 감
- IRQL 감 (너무 깊게보다 "왜 DPC/높 IRQL에서 막히는 작업이 있는지")
- 드라이버: 진입점, I/O 모델, `Mm*`/`Io*` 감
- 본인 경험: 프로세스 생성 통지 + IOCTL + 유저 에이전트 (PPT 05)

예상 질문

- 유저모드 후킹만으로 부족한 이유는?
- 커널 콜백 기반 관측의 장단·서명/HVCI 환경 제약
- 토이 드라이버에서 rename 우회가 되는 이유와 제품적 개선 (서명·해시·부모 프로세스)

3.5 컴퓨터 구조 / CPU / 메모리 / 로딩

- 캐시 라인, 정렬, false sharing (성능 문제와 연결)
- 파이프라인·스펙큘레이션은 깊게보다 사이드채널 감 정도
- MMU, TLB
- §1.1 안티디버깅과 CPU 타임스탬프

3.6 보안 관점 설계 (우회 고려)

항상 답에 넣을 4요소:

1. **공격자 목표**
2. **관측/차단 지점**
3. **우회 경로**
4. **오탐·성능·호환성**

C2. 우대요건 · 용어 카드

우대	본인 상태	준비 액션
Anti-Cheat 실무	강점	Integrity/헬스체크/보호/CI 를 설계 단위로 설명
Kernel Driver	토이+헬스체크	한계와 제품 확장 관점 준비
Assembly/호출규약/예외/디버깅	중	SEH/VEH 개념, x64 ABI 암기
IDA/Ghidra/WinDbg/x64dbg	IDA·x64dbg 강	WinDbg 기본 명령 몇 개만

Injection/Hook/Manual Map/DKOM/Rootkit/AntiDebug/AntiTamper	개념+일부 실습	**정의 한 줄 + 탐지 아이디어 한 줄**씩
TPM/Secure Boot/VBS/HVCI	약할 수 있음	각 1문장 정의 + AC에 미치는 영향
라이브 운영	실서비스 적용 경험	백신 오탐, 엔진별 이슈, 오버헤드 튜닝 사례
저수준 성능·안정성·호환	강점	Steam Deck/Unity/Unreal 연동 이야기

4.1 용어 카드 (한 줄 정의)

- **Code Injection:** 타 프로세스 주소 공간에 코드를 실행 가능하게 넣는 행위
- **API Hooking:** API 진입을 가로채 제어/관측
- **Manual Mapping:** 로더를 우회해 PE를 직접 매핑
- **DKOM:** 커널 객체를 직접 변조해 목록에서 숨김 등
- **Rootkit:** 은닉·지속을 위한 시스템 하위 조작 통칭
- **Anti-Debugging / Anti-Tamper:** 분석·변조 방해·탐지
- **TPM:** 하드웨어 신뢰 앵커
- **Secure Boot:** 부팅 체인 서명 검증
- **VBS / HVCI:** 가상화 기반 격리, 커널 코드 무결성 강화 → 서명되지 않은 커널 코드/일부 기법 제약

C3. 직무 미션형 설계 질문

공고 미션을 면접/테스트 서술형으로 바꾼 것.

1. **클라이언트 변조·메모리 조작 탐지**를 설계하라. 디스크 vs 런타임 차이는?
2. **인젝션·후킹**을 어떻게 나누어 관측하는가? 유저만으로 부족한 지점은?
3. **디버깅·분석 방해**와 **탐지 시그널 수집**을 어떻게 균형 잡는가?
4. **자동화(매크로/봇)** 는 API 차단만으로 되는가?
5. **커널 기능**을 넣을 때 안정성·호환성·HVCI를 어떻게 검토하는가?
6. **오탐**이 터지면 제품에서 어떤 스위치/텔레메트리가 필요한가?
7. **성능:** 프레임 드롭 없이 주기적 메모리 해시를 돌리려면?
8. **다중 엔진(Native/Unity/Unreal)** 연동 시 SDK 경계를 어떻게 잡는가?

본인 답 재료 (서류에서 꺼낼 것)

- 파일/메모리 체크썸, PE 파싱, ASLR·프로텍트 환경 기준점
- 중요파일 용량 체크를 별도 스레드 + 정책 on/off
- DLL/Driver 헬스체크, 언로드 탐지, 연속 미탐지 임계값

- 보호 강도 vs 로드 오버헤드 튜닝, CI/CD 보호 자동화
- 소유권 단일 경로 + move/참조 전달 (자소서 C++ 사례)

PART D. 코드 리뷰 체크리스트

> ⏱️ 직전1h · 40–48분 · 58–60분 복창
>
> 상세 함정 설명 → `### 1.11` **보강 특집**.

보안

- ☐ 버퍼 크기, 부분 읽기/쓰기, **C문자열 널**
- ☐ 경로·**정수 오버플로**·**부호/size_t**
- ☐ 권한·공유 모드 · **페이지 W+X**
- ☐ TOCTOU
- ☐ 시크릿이 로그/메모리에 남는지
- ☐ raw 구조체 **패딩/pack/정렬**

정확성

- ☐ 반환값/LastError · **CreateFile→INVALID_HANDLE_VALUE**
- ☐ 핸들/포인터 수명
- ☐ 배열 `new[]`/`delete[]`
- ☐ 가상 소멸자 · **explicit**
- ☐ 이동 후 사용 · **iterator 무효화**
- ☐ `volatile`로 스레드 동기화 흉내 (→ atomic)

성능

- ☐ 불필요 복사
- ☐ 핫패스 할당/락
- ☐ 동기 I/O on UI thread
- ☐ 과도한 가상 호출/가상 상속

Windows

- ☐ Unicode API
- ☐ 핸들 누수 · NULL/INVALID 혼동
- ☐ DllMain 규칙
- ☐ 호출 규약 mismatch (콜백)
- ☐ SEH vs C++ 예외 (크래시 경로)

PART E. 서류 · 면접 예상질문

⛔ 시험 직전 1시간 **제외** (면접용)

6.1 자기소개서 (`resume.md`)

1. "탐지 완성도보다 우회를 가정한다" — **구체 사례 하나**로 설명하라.
2. 성능·안정성·호환성·오탐을 **동시에** 본 트레이드오프 사례는?
3. CDN 라이브 업데이트 모듈의 **소유권/수명**을 코드 레벨로 설명하라. (`unique_ptr`? 누가 free?)
4. "여러 게임에 적용" — 엔진/플랫폼별로 깨진 가정은 무엇이었나?
5. 스마일게이트 AC와 KSS Game Trust Platform의 **차이**를 어떻게 이해하나? (지원 동기 검증)
6. LIG 로보틱스 경험이 AC 성능 튜닝에 **구체적으로** 어떻게 연결되나?
7. 왜 지금 이직/지원인가? (현업 AC인데 KSS인 이유)

6.2 현업 Anti-Cheat (CV + PPT 01)

1. 디스크 체크썸과 런타임 메모리 체크썸의 **비교 기준**을 어떻게 잡았나? (ASLR, 재배치, 프로텍터)
2. 블록 단위 병렬 해싱 — 동기화·부분 실패·CPU 사용률은?
3. 중요파일 용량 체크가 체크썸을 **대체**하지 못하는 이유는? 왜 같이 쓰나?
4. 헬스체크에서 **오탐**을 줄이기 위한 연속 미탐지/임계값 설계
5. 보안 DLL 강제 언로드를 유저모드에서 어디까지 볼 수 있고, 뭐가 부족한가?
6. 바이너리 보호 옵션 튜닝 시 **측정 지표**는? (콜드 스타트, 프레임 타임)
7. 백신 오탐/상용 보안 충돌을 어떻게 디버깅·완화했나?
8. 입력 데이터 수집 → AI 확장 기반: **프라이버시·성능·변조** 이슈는?
9. Steam Deck/Android까지 간 이유와 Windows 가정의 차이
10. 영업비밀을 지키는 선에서, 가장 기여도 높은 모듈은?

6.3 행위분석 플랫폼 (SW Maestro, PPT 02)

1. IAT 훅 vs Inline(JMP trampoline) — 왜 메인으로 인라인을 썼나?
2. `ConnectEx`처럼 export가 아닌 API를 어떻게 훅 포인트로 잡았나?
3. 자식 프로세스 재주입과 Process Tree — loader lock / 타이밍 이슈는?
4. 로그 자기보호·뮤텍스 — 데드락 가능성은?
5. 샌드박스 결과가 AC 제품의 "탐지"와 어떻게 다른가? (관측 vs 판정)

6.4 메신저 리버싱 (PPT 03)

1. 왜 `GetOpenFileName`이 아닐 것이라 의심했나? 가설 검증 과정은?
2. COM `IShellItem::GetDisplayName` 훅 시 수명·아파트먼트 이슈는?
3. 이 프로젝트를 AC 맥락에서 말하면? (공격 PoC가 아니라 API 선정/리버싱 역량)

6.5 게임 매크로 (PPT 04)

1. 왜 OpenCV 매칭이 아니라 `PrintWindow`가 병목/차단 포인트인가?
2. 단일 API 훅의 우회는? 그때 "제품 AC 관점" 다신호란 구체적으로?
3. 에뮬레이터 환경 탐지와 매크로 탐지의 경계는?

6.6 커널 드라이버 토이 (PPT 05)

1. `PsSetCreateProcessNotifyRoutine` 계열 흐름을 설명하라.
2. ImageFileName 문자열 매칭의 우회와 오탐
3. 유저 에이전트 ↔ IOCTL 신뢰 경계 — 인증/위조 가능성은?
4. HVCI/드라이버 서명이 있던 시절 프로젝트라면 뭐가 달라지나?
5. "학습용"과 "제품"을 나누어 말한 이유 (PPT 톤) — 면접관이 좋아함. 더 깊게.

6.7 AutoDBG / ROS (PPT 06, 석사)

1. 비결정성의 원인이 메시지 드롭·지연인 것을 어떻게 증명했나?
2. 미들웨어를 읽고 패치한 경험이 AC의 "복잡한 시스템 내부 수정"과 어떻게 같나?
3. IDA + Linux 원격 디버깅 세팅을 간단히
4. ACSAC 포스터 한 문장 요약

6.8 커리어 전반 / 약점 공격

1. 로봇 → AC 전환이 단순한 이직 아닌가?
2. 커널은 토이뿐인데 필수 요건의 커널 이해를 어떻게 증명하나?
3. WinDbg / Ghidra / VBS 경험 부족을 どう 보완 중인가?
4. C++11을 "최소"로 요구하는데, C++17/20도 쓰는가? 차이 뭐 아나?
5. 지난 지원에서 Pre-Test를 못 붙었다면 — **뭐가 부족했고 어떻게 메웠나?** (솔직+성장)

6.9 라이브 코딩/화이트보드로 나올 수 있는 변형

- 간단한 RAII 파일 핸들 클래스 작성
- `unique_ptr`로 버퍼 소유권 넘기기
- 스레드 안전한 카운터 / 잘못된 `volatile` 코드 고치기
- PE 섹션을 순회하는 의사코드
- 인라인 훅 trampoline 의사코드 (개념)
- CreateFile 래퍼의 안전한 버전 작성

답변 품질 루브릭

각 기출/예상 질문을 5점 만점으로:

점수	기준
5	정의 + 차이 + 함정/미정의 동작(Undefined Behavior) + AC/Windows 연결 + 우회/오탐
4	정의 + 차이 + 함정
3	정의는 맞지만 함정·차이를 못 말함
2	대략 알고 용어만
1	헷갈림

합격 감: 기출 10문항 평균 **4.0 이상**, C++ 문항(2–6, 10)은 **전부 4 이상**.

부록. Sir.LOIN 발췌

> ⛔ 시험 직전 1시간 **제외**
>
> 출처: Sir.LOIN 「소프트웨어 엔지니어 인터뷰 질문 목록」(공개 Asciidoc) — 경력·협업 / OS / 네트워크 / DS / 디자인·테스트 등
> 원칙: **전부 외우지 말 것.** KSS Anti-Cheat·C++·Windows에 겹치는 것만 흡수.
> 시기: **목 Pre-Test**에는 ★만. 나머지(●·○)는 Phone / Technical Fit / Culture Fit용.

범례: ★ Pre-Test·저수준 직결 · ● 직무면접 고확률 · ○ 있으면 가점 / 컬처핏

2B.1 운영체제 / 시스템 (★ 최우선 — 공고와 직결)

Sir.LOIN 「운영체제 / 논리회로 일반」에서 AC 포지션에 바로 쓰는 것:

우선	질문	KSS/AC로 연결할 답 축	상태
★	Thread 간 공유 vs Process 간 공유 — 공통점·차이	주소공간, 커널 객체(파일·공유메모리·파이프). AC: 타 프로세스 메모리 읽기 = 권한·핸들	☐
★	System call이 뭔가?	유저→커널 진입. 훅이 user API만 보면 syscall 직콜로 우회 가능	☐
★	여러 프로그램을 동시에 실행하는 원리	스케줄링, 시분할, 문맥교환. IRQL/인터럽트는 “감”만	☐
★	기계어를 읽고 실행하는 과정	fetch–decode–execute, 레지스터/메모리. 안티디버깅·싱글스텝과 연결	☐
★	Thread safety란? 어떻게 만드나?	data race 방지, mutex/atomic. AC 워커·헬스체크 공유 상태	☐

★	컴파일러 vs 인터프리터	AC 클라 = 네이티브 C++. 스크립트/매크로 자동화 탐지 맥락으로도 언급 가능	□
●	10진↔2진 변환	비트 연산 기초. 짧게 OK	□
●	소수 계산이 어려운 이유 / 해결	IEEE754, 오차. “게임 좌표/타이밍을 float로 두면?” 정도	□
●	GC란? 잘 아는 GC 하나	C++는 비GC. **실시간·결정성** 이야기로 AC/임베디드 연결	□
●	GC 언어를 원자력/자동차에 쓸 수 있나?	지연·일시정지 → 경성 실시간엔 부적합 쪽 주장 + 완화책	□
●	바이트코드 vs 기계어	Unity/IL vs Native. 보호·난독화 타겟 차이	□
●	bytecode 언어 디컴파일 취약 — 언어 안 바꾸고 완화?	난독화, 네이티브 플러그인, 서버 권한. 본인 바이너리 보호 경험	□
○	파이프(|)란?	IPC. 프로세스 간. Windows named pipe / **공유메모리 차이 → `### 3.2b`**	□
○	지금 기술/언어 장단 한 가지씩	C++: 성능·제어 vs 미정의 동작(Undefined Behavior)·복잡도. 솔직히	□

목요일 최소: 위 ★ 6문항을 각 90초 안에.

2B.2 자료구조·알고리즘·보안 기초 (★/●)

「자료 구조 및 알고리즘」에서 뽑기:

우선	질문	답 축 / 본인 연결	상태
★	암호화란? 하나 설명	대칭(AES) vs 비대칭. CDN 설정·모듈 복호화 경험과 연결	□
★	공개키 vs 비밀키	키 배포·서명·TLS 악수 한 줄	□
★	캐시란? 목적	지역성. CPU 캐시 / 디스크 / AC 정책 캐시. false sharing 언급 가능	□
●	알고리즘 성능 판단 기준	시간·공간 복잡도, 실측(프로파일). LIG·보호 오버헤드 측정 경험	□
●	캡슐화	정보 은닉. SDK 경계·Pimpl	□
●	List vs Set	중복·탐색. C++ `vector`/`set`/`unordered_set`로 바꿔 말하기	□

●	이진 탐색 최선/최악	정렬 전제, O(1)~O(log n)	□
●	손실 vs 무손실 압축	보호/패키징 맥락으로만	□
●	민감정보(전화번호) 저장 / 관리자도 모르게	해시·암호화·키 분리·접근감사. 보안 변수·메모리 잔존	□
●	RSA 키 길이 기준 / 길게만 하면 되나?	보안강도 vs 연산. “길이만으로 완벽” 아님	□
○	화면 캡처 프로그램 설계	**매크로 PPT와 직결** — PrintWindow/BitBlt, 권한, 후킹 탐지	□
○	메모리 4MiB 기기에서 10MiB 연락처	외부저장+인덱스, 페이징, 압축. 임베디드 사고력	□
○	순환 Queue / 무한 순환 검출	visited 세트, Floyd. 그래프	□

2B.3 디자인 · 테스트 · 품질 (● — Pre-Test보다 면접)

「디자인 및 테스트」+「경력·협업」에서 AC 엔지니어답게:

우선	질문	답 축 / 본인 연결	상태
★	Singleton 장단	전역상태, 테스트 어려움, 수명. **본인 PPT/코드의 static instance 위험**과 연결	□
★	Mutable vs Immutable	공유·스레드. const 멤버, 방어적 복사	□
★	Callback/Closure 주의점	수명, `this`, 언로드 후 호출 — DLL·훅과 동일 문제	□
●	좋은 테스트의 가장 중요한 요소	결정성·빠른 피드백·실패 원인 명확. AC는 오탐 시나리오 테스트	□
●	Unit / Integration / E2E / Smoke 구분	본인 CI에 QA 빌드·보호 파이프라인 연결	□
●	CI가 왜 중요한가?	자소서: 보호·검증·배포 자동화. “수동 프로텍트” 제거	□
●	코드 리뷰에서 무엇을 보나?	수명·레이스·경계·성능·우회면 (AC 렌즈)	□
●	코드 품질 기준	정확성 > 보안 가정 > 가독성 > 성능. 과장 금지	□
●	파라미터 객체를 메소드가 바꾸면?	const&, 복사, 문서화. 부작용 = API 계약	□
●	상속으로 중복 제거는 나쁜가?	구성(composition) 선호, LSP. “코드 재사용 ≠ 상속”	□

●	memory leak 판단·해결	CRT/Win 힙 도구, 수명, 콜백 미해제. 게임 장시간 플레이	□
●	외부 API가 예고 없이 바뀌어도 크래시 없게	버전 협약, 방어적 파싱, 기능 플래그 — 엔진/OS 업데이트 비유	□
●	MSA vs Monolith 선택 기준	AC는 “클라 모듈 + 서버 탐지” 경계로 재해석	□
○	Acceptance 등 테스트 피라미드 본인 기준	한 장으로 그려 말하기	□
○	blackbox vs whitebox	치트 샘플 = 블랙박스, 모듈 단위 = 화이트박스	□

경력·협업 (컬처핏 / 폰인터뷰):

우선	질문	준비 메모	상태
●	3년 후 / 10년 후 모습	KSS·Game Trust·클라이언트 보안 깊이	□
●	수습 기간 능력 증명	첫 모듈 온보딩, 오탐↓, 문서화	□
●	업무 중 다른 요청 → 조율	우선순위·리스크·일정 재합의	□
●	납득 안 가는 일 / 갈등	근거·대안· Escalate	□
●	왜 소프트웨어 엔지니어인가	분석→제품 방어 스토리 (자소서와 일치)	□
●	XX 기술을 안 쓰는 회사 설득	문제·비용·리스크·PoC. “취향” 금지	□
○	Agile 생각·단점	보안은 스프린트만으로 안 끝나는 부채	□
○	스케줄 시간 비용 산정	불확실성 버퍼, 스파이크	□
○	GPL이 뭔가?	카피레프트 감. 상용 AC와 라이선스 주의	□

2B.4 네트워크 (○ — 알고 있으면 가점)

AC 클라는 주력이 아니지만, CDN 업데이트·로그 전송·TLS는 닿음.

우선	질문	답 축	상태
●	HTTPS URL 입력 후 일어나는 일	DNS→TCP/TLS→HTTP. 너무 깊게 말고 단계	□
●	TLS가 어떻게 동작하나 (개요)	인증서, 키교환, 대칭세션	□
●	HTTP Stateless인데 로그인은?	쿠키·토큰·세션	□
○	Socket 직통신 대비 HTTP를 쓰는 이유	미들웨어·캐시·표준·방화벽	□

○	HTTP/3이 UDP(QUIC)인 이유	헤드오브라인, 연결 마이그레이션	□
○	HTTP로 바이너리 전송	multipart, raw body, base64 트레이드오프	□

스킵 권장(목 Pre-Test): OSI 7계층 암기 대회, 소켓 크롤러 구현 상세.

2B.5 데이터베이스 — 대부분 스킵

도축장 모델링·JPA·샤딩 등은 **KSS AC Pre-Test 우선순위 밖**.
시간 남으면 아래만:

우선	질문	왜 Minimal인가	상태
○	Index로 빠른 검색 (할머니도 이해하게)	B-Tree 전화번호부 비유. 서버 탐지 로그 조회감	□
○	낙관적 vs 비관적 락	버전/충돌 vs 선점. 클라 동기화와 혼동 금지	□

2B.6 모바일·기타 언어 — 선택만

본인 CV에 Android AC가 있으므로:

우선	질문	답 축	상태
●	(Android) 비동기 작업 결과를 UI에?	메인 스레드 마샬링. Native/NDK면 별도	□
○	Memory leak 유발 코딩 패턴 (Android)	Context 누수, 정적 뷰 참조	□
○	HAL이란?	하드웨어 추상. 깊게 X	□

스킵: Kotlin/Dart/Flutter/iOS/Spring/JPA/Node 세부 — 시간 대비 효율 낮음.
Python GIL·`yield`는 "알고 있다" 수준이면 충분.

2B.7 Sir.LOIN → 이미 §2A/기출과 겹치는 것 (중복 학습 금지)

이미 준비한 것과 **같은 주제**면 Sir.LOIN 원문 대신 기존 노트로:

Sir.LOIN 쪽	이 문서
Thread safety	§ 2A D급, § 1.3 volatile
Callback 수명	§ 2A B10–B11, H04
Singleton 테스트 어려움	§ 2A + 클래스리뷰 static instance

Mutable/Immutable	const, §2A
CI 중요성	자소서 Build&Ops, §6
암호화/공개키	위 2B.2 + 현업 복호화 모듈
화면 캡처 설계	PPT 04 매크로
System call	§1.7–1.8, §3.4

더 볼 자료 (시간 남을 때)

- C++ Core Guidelines: ES.50, R., *CP.* 일부
- Microsoft Docs: x64 calling convention, PE format overview
- Windows Internals 요약: Process/Thread/Memory/DLL chapters
- 본인 PPT를 **발표 없이 읽어도 이해되는지** 다시 점검 (면접에서 화면 공유될 수 있음)

Last updated: 2026-10-26 — KSS Pre-Test prep (Thu exam / 3-day plan)

Reorganized: 2026-10-26 — topic index + PART A~E